
基于 LLM 引导的目标演化 的按需出行系统分层优化

易 Zhang^{1*} 雨申 Long² 云 Ni³ 丽萍 Huang¹ 小红 Wang¹ 君 Liu⁴
1 新加坡科学技术研究局 2 摩根士丹利亚洲私人有限公司 3 Onto
Innovation Inc. 4 英国兰卡斯特大学计算与通信学院
{Zhang_Yi, Huang_Liping,
Wang_Xiaohong}@a-star.edu.sg
Yushen.Long@morganstanley.com,
Yun.Ni@ontoinnovation.com
j.liu81@lancaster.ac.uk

摘要

在线网约车平台旨在提供高效的按需出行服务，但往往面临平衡动态和空间异构供需的挑战。现有方法通常分为两类：强化学习（RL）方法，其存在数据效率低下、现实世界动态建模过于简化以及难以执行操作约束的问题；或分解的在线优化方法，这些方法依赖于手动设计的高层目标，缺乏对低层路由动态的了解。为了解决这个问题，我们提出了一种新颖的混合框架，将大型语言模型（LLM）与动态分层系统中的数学优化相结合：（1）它无需训练，无需像RL那样对大规模交互数据；（2）它利用LLM通过自适应生成高层目标来弥补问题分解造成的认知限制。在此框架内，LLM充当元优化器，产生语义启发法，指导负责约束执行和实时决策执行的低层优化器。这些启发式方法通过由和声搜索驱动的闭环演化过程进行完善，该过程根据优化层的可行性和性能反馈迭代地调整 LLM 提示词。基于纽约和芝加哥出租车数据集的场景进行的大量实验证明了我们方法的有效性，与最先进的基线相比，平均提高了 16%。

1 引言

网约车服务等按需出行平台已成为重要的城市交通基础设施，通过持续执行订单调度和车辆路线两个核心决策流程来解决供需失衡问题 [1, 2, 3, 4, 5, 6]。这种顺序决策涉及解决时空约束下的复杂组合优化问题，确保在不断变化的城市环境中及时高效地提供服务。

首先，强化学习方法 [7, 5, 8, 9, 10, 11] 被用来学习解决问题的策略。这些方法可以通过考虑驾驶员和乘客的未来轨迹来获得长期回报。然而，RL 方法通常需要大量的训练数据和交互才能有效学习。训练可能不稳定，并且执行硬约束具有挑战性。此外，学习到的策略在其训练分布之外可能会出现不可预测的行为。

* 通讯作者

其次，部署了两阶段优化框架来分解问题以降低复杂性。通常，高层阶段通过混合整数线性规划（MILP）解决供需平衡，例如，批量匹配 [2 , 3 , 1 2]，而低层阶段则侧重于调度或路由，例如，基于图的搜索 [1 3 , 1 4]。然而，高层目标通常是在不充分了解低层动态的情况下手动设计的，这可能会导致次优决策。这种脱节可能会导致目标不一致，无法完全反映真实的运营限制，最终导致效率低下，例如等待时间增加或闲置距离增加。第三，大型语言模型提供了一种新的范式：它们嵌入的组合推理和城市流动性的先验可以提高适应性。然而，当前的应用主要关注静态优化设置 [1 5、1 6、1 7、1 8、1 9]。当应用于具有实时状态转换的动态网约车系统时，这些方法面临两个关键限制：（1）缺乏与不断发展的系统状态相一致的迭代细化，以及（2）缺乏求解器集成，导致提出的建议可能违反约束或降低解决方案质量。为了解决上述问题，我们首次将 LLM 与动态顺序决策系统的数学优化集成。具体来说，我们提出了一种混合的 LLM 优化器框架，该框架分层分解问题，仅在存在人类专业知识瓶颈的情况下策略性地嵌入 LLM：（1）LLM 作为元目标设计器：在优化求解器的可行性反馈的指导下，通过基于提示词的和声搜索 [2 0 , 2 1] 动态演化高层目标。（2）优化器作为约束执行器：以数学严谨性解决低层路由，确保实时可行性。（3）启发式作为提示词进化器：利用和声搜索算法迭代地细化 LLM 提示词，在优化器反馈的指导下自适应探索并收敛到有效的元目标。该框架无需培训，无需基于 RL 的方法所需的大量数据或交互。同时，通过利用 LLM 自适应地演化高层目标，它可以减轻分解优化中手动设计引入的次优性，使高层决策与下游动态更加紧密地结合起来。

我们的框架将每个决策步骤分层分解为两个层次：高层模块负责根据实时空间配置和预期的供需不平衡将乘客分配给出租车，而低层模块解决每辆出租车的路线或访问顺序问题，以最大限度地减少时空约束下的乘客等待时间。为了解决分解引起的部分可观测性挑战（高层模型缺乏对下游路由动态的远见），我们采用 LLM 作为元启发式设计器，利用其对城市流动模式的隐式理解来自适应地细化高层目标。如图 1 所示，LLM 生成高层分配目标，充当优化循环内的语义指南。这些目标被嵌入到闭环进化过程中，每个模拟时期都会评估其 fitness。进化机制以和声搜索算法为指导，迭代细化 LLM 提示词空间以提高客观质量。这种反馈驱动机制使 LLM 生成的启发式方法能够随着时间的推移进行调整和改进，将 LLM 的语义丰富性与传统优化的结构稳健性相结合。通过将 LLM 作为语义目标生成器集成到分层优化循环中，我们的框架实现了静态运筹学（OR）公式中所缺乏的动态适应性，同时避免了基于 RL 的方法的数据效率低下。

我们的贡献总结如下：1）我们提出了一种混合的 LLM + 优化器范式，其中 LLM 充当元优化器，通过基于提示词的进化来进化高层目标，而数学优化求解器则保证约束满足和数值严谨性。据我们所知，这是第一个利用 LLM 的功能与顺序动态决策系统的优化求解器相结合的工作。2）我们引入了一种具有三个新颖算子（随机推理、启发式改进和创新生成）的和声搜索算法，以使用数学求解器的反馈迭代地细化 LLM 生成的目标。3）对纽约和芝加哥出租车数据集得出的各种场景进行的广泛实验证明了我们方法的有效性，与最先进的基线相比，将乘客等待时间减少了约 1 6 %。

2 相关工作

网约车中的强化学习采用了基于 RL 的方法，使系统能够从历史数据中学习，并通过考虑预期来优化决策

LLM 用于运筹学问题 最近的研究探索了使用 LLM 来自动解决 OR 问题，方法分为两大类：（1）自动化数学模型制定以减少对领域专业知识的依赖 [18, 17, 25, 26, 19, 27]。已经提出了任务分配、小样本学习和专家链等方法来指导程序化 LLM 提示词 [28, 29, 30, 17, 31, 32]。（2）直接查询 LLM 的算法代码以找到 OR 问题的可行解决方案 [15, 33, 34, 35, 36]。FunSearch [15] 利用 LLM 的生成能力提出候选项目，然后对其进行严格评估以识别高质量的解决方案。EoH 框架 [16] 将启发式概念编码为自然语言“思想”，然后由 LLM 将其翻译为可执行代码。此外，它还采用进化算法来迭代地完善和进化提示词。然而，据我们所知，还没有研究将 LLM 与动态决策系统背景下的优化公式相结合。

3 方法

3.1 动态分层优化问题

我们提出了一种新颖的网约车系统问题表述（附录 A.6），并对相关搜索空间复杂性（附录 A.5）进行了形式化分析，共同推动将原始任务分解为两个易于处理的子问题 - 涉及任务分配和车辆路径中的顺序决策。这些子问题集成在由 LLM 组件增强的分层框架内。

第一层分配问题作为分配问题，我们的目标不仅仅是确定空闲出租车的下一个直接乘客。相反，我们在整个系统快照中制定全面的分配，确保为每位乘客提供服务并分配至最多一辆出租车。省略在第一层捕获出租车动态的额外变量可以显著降低问题的复杂性。然而，传统方法采用手工设计的目标（左下），但缺乏对低层路由动态的认识可能会导致系统性能不佳：

$$\begin{array}{ll} \text{分钟 } J_{1st} & J_{dist}^{p,v} y^{pv} \quad (TROP_{Sv} + TRDP_{Sv} \text{ 最小 } \Phi_t = LLM(S_t, H_{t-1})) \\ \text{分钟 } J_{1st} & J_{time}^{p,v} y^{pv} \quad (T_{ps} \text{ 分钟 } J_{1st} = LLM \text{ 精炼 } s.t. y^{pv} = \\ & J_{util}^{p,v} y^{pv} \quad (利用率) \end{array}$$

注：Op、Dp：旅客 p 的始发地、目的地点； Sv：Taxi v 起始位置。 Trij：行程时间 i→j； T p：旅客请求时间； t Sv：出租车 v 开始时间。 ypv ∈ {0, 1}：二元赋值变量。

这些目标分量可以通过加权求和线性组合： J1st = αJdist1st + βJtime1st + γJutil1st，其中固定 weights α、β、γ 权衡距离（车辆公里）、时间对齐（估计等待时间）和利用率公平。虽然这种短视的方法缺乏对低层动态的可见性，但我们的框架用 LLM 生成的 Φt = LLM(S t, H t-1) 取代了手工制作的目标，其中 S t 编码车辆位置和待处理请求，H t-1 捕获历史拥堵和分配。通过将潜在的城市动态嵌入到 MILP 目标中，LLM 弥合了分配和路由之间的差距，而无需明确的动态建模。

二级排序问题解决第一层问题并将乘客分配给每辆出租车后，就可以为每辆出租车独立解决第二层问题，消除索引 v。在下面的公式中，Op 和 Dp 代表乘客 p 的出发地和目的地，而 S 和 E 代表出租车的源站和宿站。 ~ t 是预计到达时间。上车服务点集合 P 由 { (p , Op) } 组成，下车服务点集合 D 由 { (p , Dp) } 组成。

约束 (1b) - (1d) 初始化位置和到达时间分配，其中 xij 是指示链路选择的二进制变量。 ARi 和 DPi 表示位置 i 的到达时间和离开时间。约束 (1e) - (1f) 强制流量保护，确保相等的传入和传出链路。约束 (1g) - (1m) 定义系统动力学：(1g) - (1h) 将出租车到达下车点的时间确定为接载出发时间和行驶持续时间的总和，而 (1j) - (1k) 对新接载执行相同的操作。

出发时间不得早于到达时间 $(1i)$ 、 $(1l)$ ，并且必须大于或等于乘客请求时间 $(1m)$ 。目标 $(1a)$ 最大限度地减少乘客的总等待时间。

$$\text{最小化 } J2nd = p \cdot (DP(p, Op) - T_p) \quad (1a)$$

$$\begin{aligned} \sum_{i \in \mathcal{P}} x_{S,i} &= 1 & (1b) \\ ARS &= \sim t \quad (1c) & X \\ \sum_{i \in \mathcal{P}} x_{ij} &= \sum_{q \in \mathcal{P}} x_{jq}, \forall j \in \mathcal{D} & (1e) \\ \sum_{j \in \mathcal{D}} x_{ji} &= \sum_{k \in \mathcal{D}} x_{ik}, \forall i \in \mathcal{P} & (1f) \\ &+ M(1 - x(p, Op), (p, Dp)) \quad (1g) \end{aligned}$$

$$\begin{aligned} &-M(1 - x(p, Op), (p, Dp)) \quad (1h) \\ DP(p, Dp) &\geq AR(p, Dp) \quad (1i) \\ AR(p', Op') &\leq DP(p, Dp) + \\ &TRDPpOp' + M(1 - x(p, Dp), (p', Op')) \quad (1j) \\ AR(p', Op') &\geq DP(p, Dp) + \\ &-M(1 - x(p, Dp), (p', Op')) \quad (1k) \\ DP(p', Op') &\geq AR(p', Op') \quad (1l) \\ DP(p', Op') &\geq T_{p'} \quad (1m) \end{aligned}$$

3.2 LLM - 优化器交互协议

3.2.1 场景提示词设置

场景提示词 P_{sce} 为 LLM 提供有关问题、分配的角色和特定任务的上下文知识。提示词结构通过三个嵌套层形式化： $P_{sce} = P_{sys} \oplus P_{geo} \oplus P_{model}$ 。 (1) P_{sys} 定义了 LLM 作为自适应目标设计器的角色，将输出与求解器兼容的模板对齐； (2) P_{geo} 通过曼哈顿 / 芝加哥区域图和 OD 矩阵编码地理空间语义； (3) P_{model} 提供类蓝图以实现变量感知目标制定。与仅指定输入格式或参数类型的 P_{data} 相比， P_{model} 确保 LLM 生成的目标尊重类结构和求解器约束。附录 A.9.1 提供了更多详细信息。

3.2.2 动态环境反馈

由于每次模拟运行都涉及多个决策步骤，因此有两种方法可以使用 LLM 进行推理，如图 2 所示： (1)

在测试开始时进行一次查询，其中生成的目标对于所有后续步骤保持固定，或 (2) 在测试的每个步骤中进行查询。这些对应于两种推理策略。从控制的角度来看，

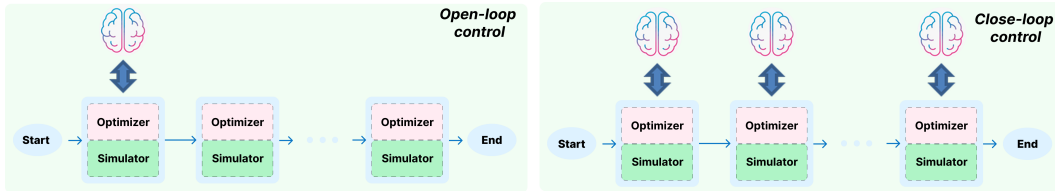


图 2：单次模拟运行中的推理策略： (1) 开环控制，在测试开始时发生一次性查询。 (2) 闭环控制，在测试的每一步都发生查询。

第二种方法发生多个查询，结合了来自模拟器的实时环境反馈，允许 LLM 动态生成更新的目标。这形成了闭环控制系统，通常对于自适应决策更有效。因此，我们在推理策略中采用了这种方法。为了实现闭环方法，必须将最新的环境状态纳入提示词中，确保 LLM 保持上下文感知。具体来说，我们定义提示词文本如下。

动态状态流 P_{dyn} ：在时间步 t ，提示词上下文 $P_{t\ dyn} = \{(veh_t, pass_t)\}$ ，其中 $veh_t \in \mathbb{R}^{|veh| \times 2}$ 表示车辆状态向量，例如位置和到达时间。
 $pass_t \in \mathbb{R}^{|pass| \times 3}$ 表示需求张量，包括起点 - 目的地对和请求时间。

3.2.3 进化式提示词优化

为了通过体验式学习促进 LLM 中的推理，我们使用进化算法（EA）迭代优化提示词群。与个体为参数向量的传统 EA 不同，这里的每个提示词都被视为一个不断发展的实体， $fitness$ 通过基于模拟的推理分数进行评估。快速进化是由和声搜索（HS）算法生成的启发式方法引导的，这是一种在经典优化中成熟的方法。

和声搜索算法 HS 模拟音乐演奏的过程，旨在达到最佳的和声状态。如算法 1 所示，其演化由两个关键参数控制：和声记忆考虑率（HMC R）和音调调整率（PAR）。HMC R 确定下一次迭代是否涉及探索分支（引入新解决方案）或开发分支（完善现有解决方案）。如果选择利用，PAR 会以一定的概率促进局部调整，从而允许微调修改。即时进化过程通过种群矩阵 $P \in \mathbb{R}^{I \times N \times T}$ 形式化，其中 I 、 N 、 T 表示迭代次数、种群大小和模拟步骤。
 $P_i = [Y(i)_1, \dots, Y(i)_N]$ ， $Y(i)_k = [X(i)_k, f(i)_k]$ ，其中 $X(i)_k$ 表示第 i 次迭代中的第 k 个个体， $f(i)_k$ 为其适应度值，同时， $X(i)_k = [x(i)_k(1), \dots, x(i)_k(T)]$ 。 $fitness$ 景观定义为： $f(X) = E_{\tau} D[R(E(X, \tau))]$ ，其中 E 代表模拟环境， R 为轨迹 τ 上的适应度代价。我们通过概率转移矩阵调整 HS 算法：

$$\Pi = \text{探索 (如果 } \alpha > \text{HMC R)} \quad \text{开发 (如果 } \alpha \leq \text{HMC R)} \quad (2)$$

其中 $\alpha \sim U(0, 1)$ 。开发阶段采用双重细化策略： $We_{exploit} = \lambda We_{heuristic} + (1 - \lambda) Win_{novative}$ ，其中 λ 由 PAR 控制。

如图 3 所示，改变 α 和 λ 的值会导致选择三种不同的提示词组合之一。每个组合由四个组件组成：场景块 $Psce$ 、动态约束块 P_{dyn} 、表 1 中定义的操作符块和格式限制块 $P_{restriction}$ ，附录 A.9.3 中提供了进一步的实现细节。

表 1：LLM 驱动的目标生成的三个算子。方程和示例提示词说明了每个算子的机制。

随机推理（W1）	启发式改进（W2）	创新生成（W3）
如果 LLM 从父提示词中汲取灵感，制定了新颖的提示词。 $X_{new} = LLM(\emptyset, Psce)$ $X_{new} = LLM(X_{parent}, Psce, I1)$ $X_{new} = LLM(X_{parent}, Psce, I2)$	提示词示例：“通过 [指令 block1] 开发改进的目标函数。”	提示词示例：“通过 [指令 block2] 重新创建上次运行的目标函数。”

表 1 列出了我们的三个进化运算符。指令块“[指令block1]”和“[指令block2]”包含分配给各自运算符的特定指令，由DeepSeek-R1 [37][38]生成。详细解释见附录 A.9.2。

在整个模拟运行过程中，在不同的时间步发出多个查询，因此，在算法 1 中共同选择不同时间步的操作符类别。在这个阶段，仅选择父提示词($X_i, fitness_i$)和操作符类别，同时合并附加组件，例如场景块、动态块和格式块，以在执行模拟步骤时构建完整的提示词。附录 A.4 提供了该过程的详细全循环算法。在算法 1 中，函数INDIVIDUALSELECT试图从旧种群中选择一个精英父母（具有良好fitness的个体）。每个个体由三个部分组成：跨越T个时间步长的动态环境输入；相应的LLM响应目标，也跨越T个时间步长；以及在T范围内评估的最终适应度值。随着范围T的增加，每个父项的上下文长度变得明显更大。为了缓解不断增长的令牌大小，使用函数TOKENSELECT来仅保留最关键的组件，特别是目标部分，然后将其与适应度值组合以构造最终的父提示词。

4 实验

4.1 实验设置

数据集和基线 表 2 中的 9 个测试场景是使用纽约出租车数据集构建的 [39]。我们还在芝加哥出租车数据集 [40] 上进行了测试，实验和详细信息在附录 4.2 的表 3 中提供。附录 A.3 给出了情景生成过程的全面描述。所有基线方法的详细信息均在附录 A.2 中提供。

评估指标在模拟器中，计算每个乘客的等待时间，并使用平均等待时间作为评估指标，如表 2 和表 3 所示。因此，采用二级优化框架的方法应将乘客等待时间作为二级排序问题的目标。类似地，在 RL 方法中，最小化总时间作为模型训练的奖励信号。对于基于 LLM 的方法，提供提示词以指示 LLM 尽量减少乘客等待时间。

实现细节在我们的实验设置中，我们通过 Hugging Face 平台 API 使用 DeepSeek-R1-Distill-Qwen-32B [41] 模型作为所有基于 LLM 的方法的默认大语言模型，这使我们能够评估我们的方法在较小的 LLM 上的适应性，从而突出其潜在应用。温度参数配置为 0.9。基于 LLM 的方法对于每个场景都执行 3 次，这 3 次运行的平均值在表 2 和表 3 中报告。FunSearch 在 20 次迭代下执行。EOH 和我们的方法都采用 10 次迭代，总体规模为

5。所有基于优化器的方法，无论是手动目标还是我们的自适应目标方法，都采用优化求解器 Gurobi [42] 来解决在第 13 代智能英特尔酷睿 i9-13900KF × 32 CPU 高达 5.80 GHz 和 RAM 32GB 的 PC 上运行的问题。

4.2 实验比较

曼哈顿出租车数据集的基线比较 表 2 展示了曼哈顿市中心的实验结果。我们的方法在 9 个场景中的 7 个场景中优于最佳基线 (FunSearch)（不包括小规模案例 1 和 3），并且所有场景的平均改进超过 14%

场景。在大规模场景（8 和 9）中，与最强基线（Full RL）相比，它进一步将平均乘客延误减少了 40% 以上。可以发现三个主要发现：

- 手动目标限制：虽然集成距离、时间和利用率信号（距离 $\times$ 时间 $\times$ 利用率）的复合目标函数优于单目标变体（例如，场景 3 中为 5.77 分钟 vs. 14.88 分钟），但它们的有效性随着问题规模的增加而恶化：场景 9 中的等待时间增加到 9.27 分钟（P200 - C100 - T1200），表明可扩展性有限。
- 基于 RL 和仅 LLM 的方法：RL 在低复杂度设置中表现良好（场景 1 中的 1.27 分钟和 3.10 分钟），但在较大场景中遇到奖励稀疏和探索效率低下的问题，尤其是第一个 RL 方法（场景 9 中的 10.35 分钟）。虽然基于 LLM 的 EoH 在动态适应方面遇到了困难（场景 9 中的 10.27 分钟），但 FunSearch 的启发式方法显示出更好的泛化能力（8.98 分钟）。然而，这两种方法都缺乏动态的、按时间间隔的反馈和低层优化器来提高解决方案的质量，导致在大型情况下与我们的混合方法相比性能不理想。
- 混合 LLM + 优化器方法：通过将 LLM 适应性与优化器精度相结合，我们的混合框架可提供一致的性能增益，特别是在高需求、长期条件下（场景 9 中为 4.01 分钟），比完整 RL（6.82 分钟）提高约 40%。性能差距随着规模的扩大而扩大，凸显了动态目标制定和闭环 LLM 优化器交互的重要性。

我们的方法通过将 LLM 驱动的语义推理与求解器强制的数学严谨性相结合而取得成功。LLM 通过基于提示词的更新迭代地细化高层目标，克服了缺乏低层约束意识的传统系统中静态目标的僵化。同时，求解器可确保时空可行性和大规模数值精度，而不依赖于 RL 中要求的大型训练数据集。这种闭环机制通过 LLM 调整目标并通过求解器强制执行可行性，有效平衡探索和可处理性，在按需出行基准上实现最先进的性能。附录 A.1.1 提供了对芝加哥数据集的进一步分析和实验。

表 2：纽约出租车数据集上不同优化方法和场景的平均乘客等待时间（分钟）

类别 方法 场景											
		1	2	3	4	5	6	7	8	9	
手动目标	Distance [†] [2]	6.51	14.78	14.88	11.42	24.67	22.32	18.09	34.98	28.04	
	Distance $\times$ Utilization*	5.02	7.38	7.94	6.96	9.07	9.89	7.06	9.24	11.59	
	Temporal [‡] $\times$ Utilization* [24]	10.09	8.81	9.93	9.74	11.48	15.23	10.97	13.26	18.99	
	Distance $\times$ Temporal $\times$ Utilization	3.32	4.99	5.77	4.37	6.06	6.72	5.19	7.07	9.27	
RL Methods	Default* + RL-Seq [§] [23]	3.10	4.37	4.90	4.92	6.63	6.59	7.48	7.51	10.35	
	Full RL [6]	1.27	2.35	3.98	2.42	3.80	4.83	3.20	4.78	6.82	
LLM Methods	FunSearch [15]	0.77	1.74	1.55	5.29	2.95	5.43	5.78	5.79	10.27	
	EoH [16]	1.64	1.58	2.79	3.04	4.68	5.71	3.87	5.55	8.98	
混合方法 (LLM + 优化器) 我们的		1.55	1.37	2.50	1.89	2.59	4.14	3.10	2.25	4.01	

距离：车辆起始位置到乘客上下车地点之间的行驶时间。时间：车辆非空闲时间与乘客请求时间之间的差距。利用率：出租车服务效率（车辆 / 请求）。默认：在第一层分配优化中使用默认目标（距离 $\times$ 时间 $\times$ 利用率）。RL - Seq：采用强化学习方法解决二级排序问题。

芝加哥出租车数据集的基线比较
由于与曼哈顿相比，芝加哥区域的空间范围相对较大，加上乘车请求分布高度不平衡（详见附录 A.3），芝加哥的平均出发地 - 目的地（OD）出行时间明显更长。行程距离的增加导致表 3 中各个场景的乘客等待时间总体较高。表 3 展示了我们的框架在芝加哥配送条件上的一贯优势，在大规模场景中平均乘客等待时间减少了 18%（例如，场景 7 中为 10.79 分钟 vs. 14.35 分钟，场景 8 中为 14.51 分钟 vs. 15.93 分钟，场景 9 中为 15.37 分钟，而场景 9 中为 19.79 分钟）。与最佳基线（FunSearch）相比，我们的方法在 9 个场景中的 8 个中表现出色，小规模案例 3 中略有例外，并且在所有案例中平均改进超过 18%。可以发现三个关键见解：

- 手动目标限制：复合目标（距离 × 时间 × 利用率）在规模下严重退化，场景 9 中的延迟升级至 25.01 分钟（而我们提出的方法为 15.37 分钟）。当出行分布明显不平衡时，如附录 A.3 所示，仅针对出行距离进行优化的单目标变体在高需求条件下表现不佳，导致极端延误（场景 8 中为 145.14 分钟）。
- 基于 RL 和仅 LLM 的方法：完整的 RL 优于 Default+RL-Seq (20.03 分钟 vs. 24.63 分钟)，但仍然不如我们提出的方法，与最佳性能方法（15.37 分钟）相比，差距为 4.66 分钟。这强调了 RL 对奖励设计和训练数据覆盖范围的敏感性。仅 LLM 的方法表现出不一致的适应。具体来说，FunSearch 在较小规模的场景（场景 1 - 3）中优于 EoH，而 EoH 在较大规模的场景（场景 7 - 9）中取得了更好的结果。然而，这两种方法都缺乏动态反馈机制和细粒度优化层来确保解决方案质量，导致相对于我们提出的混合方法性能较差。
- 混合 LLM + 优化器方法：我们的框架在所有场景中提供强大的性能，其中一些案例显示出显著的收益。例如，在场景 4 中，我们的方法实现了 8.40 分钟的平均延迟，而完整 RL 的平均延迟为 12.05 分钟；在场景 7 中，使用完整 RL 时为 10.79 分钟，而使用完整 RL 则为 14.35 分钟；在场景 9 中，EoH 为 15.37 分钟，而 EoH 为 19.79 分钟。这些改进均超过 20%，凸显了闭环适应在动态和高需求环境中的重要性。

这些结果概括了曼哈顿数据集的发现（表 2），证明了我们的方法对不同城市布局和需求分布的适应性。大规模场景中的优势强调了现实世界的乘车系统迭代 LLM 优化器交互的必要性。

表 3：芝加哥出租车数据集上不同优化方法和场景的平均乘客等待时间（分钟）

类别	方法	场景										
			1	2	3	4	5	6	7	8	9	
手动目标	Distance [†] [2]		17.52	35.87	47.85	41.08	80.96	121.41	49.99	145.14	102.44	
	Distance × Utilization*		11.60	15.34	18.35	15.70	18.50	22.77	19.07	22.74	30.38	
	Temporal [‡] × Utilization* [24]		17.07	18.15	16.90	17.33	19.74	23.96	18.51	22.09	26.96	
	Distance × Temporal × Utilization		9.54	13.79	18.19	11.81	16.71	20.74	17.21	20.68	25.01	
RL Methods	Default* + RL-Seq [°] [23]		10.94	10.87	15.37	19.27	18.68	21.57	24.02	24.86	24.63	
	Full RL [6]		10.83	13.43	15.40	12.05	14.43	16.50	14.35	15.93	20.03	
LLM Methods	FunSearch [15]		9.03	10.70	10.87	21.70	12.82	15.35	18.29	17.42	21.74	
	EoH [16]		10.43	10.83	13.05	13.53	14.45	17.16	15.01	16.93	19.79	
混合方法（LLM + 优化器）			我们的	8.65	9.58	11.30	8.40	12.32	14.96	10.79	14.51	15.37

距离：车辆起始位置到乘客上下车地点之间的行驶时间。时间：车辆非空闲时间与乘客请求时间之间的差距。利用率：出租车服务效率（车辆 / 请求）。默认：在第一层分配优化中使用默认目标（距离 × 时间 × 利用率）。RL-Seq：采用强化学习方法解决二级排序问题。

4.3 消融实验

场景提示词组合结合编码优化模型的结构蓝图的 Pmodel，显著减少了等待时间（例如，P70 - C60 - T600 为 2.86 ± 0.29 分钟），但由于

表 4：不同提示词组合下的平均等待时间（分钟）

Psys	UPgeo	UPdata	9.93 ± 0.00	5.86 ± 0.00
5.29 ± 0.00	6.46 ± 0.00	Psys	UPgeo	
UPmodel	8.72 ± 0.91	2.86 ± 0.29	3.07 ± 0.89	4.54 ± 0.36
+Prestriction	8.16 ± 1.48	4.12 ± 0.83	2.59 ± 0.52	2.25 ± 0.05
注：平均值±标准值运行偏差。粗体：每个场景的最佳结果（P = 乘客，C = 出租车，T = 时间）。提示词变体：- Psys UPgeo UPdata：系统+地理+数据结构（第 3.2.1 节）。- Psys UPgeo UPmodel：系统+地理+模型结构（第 3.2.1 节）。- +Prestriction 表示 Psys UPgeo UPmodel +Prestriction：系统+地理+模型+Gurobi-compatible 约束（附录 A.9.3）。Pdata 定义变量格式，Pmodel 指定完整的 MILP 结构。				

LLM-生成功能多样性。进一步积分 `Prestriction (Gurobi-compatible` 约束) 可实现三种组合中最低的 `costs` (例如, `P130_C80_T1200` 为 2.25 ± 0.05 分钟)。

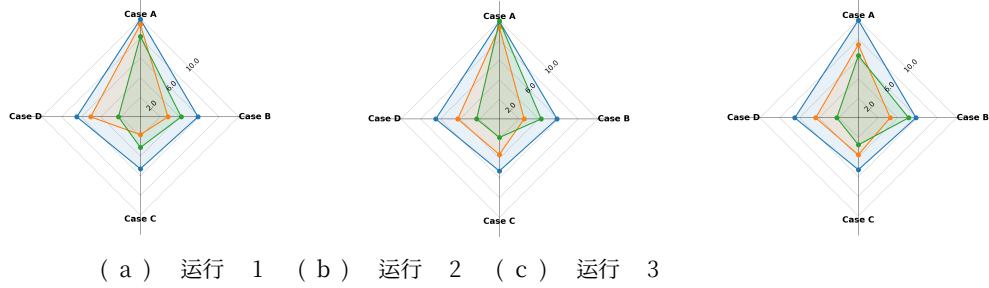


图 4：不同组成和班次下乘客的平均等待时间（分钟）。案例：A (`P50_C30_T300`)、B (`P70_C60_T600`)、C (`P100_C80_T900`) 和 D (`P130_C80_T1200`)。蓝线 (`Psys ∪ Pgeo ∪ Pdata`)、橙线 (`Psys ∪ Pgeo ∪ Pmodel`)、绿线 (`Psys ∪ Pgeo ∪ Pdata ∪ Prestriction`)。

图 4 给出了与表 4 对应的平均乘客等待时间。当问题规模较小时（案例 A 和案例 B），`Pmodel` 就足够了，但随着复杂性的增加，`Prestriction` 就变得至关重要。这与表 6 一致，其中较大的场景对于缺少约束的组合表现出较高的错误率（e.g., 没有 `Prestriction` 的 `P130_C80_T1200` 中的错误率为 36.7% ），需要显式形式化来稳定 LLM 输出。这强调了结构化任务基础（模型定义）和约束形式化对于可靠的 LLM 驱动优化的必要性。

动态环境反馈影响如表 5 所示，我们在两种查询机制上进行实验——第一个时间步上单次，每个时间步上多次——在场景 `P130_C80_T1200` 中，提供了最后一次迭代每次运行的最终结果。显然，动态多次查询比单次查询表现出更优越的性能。动态方法受益于与 LLM 的频繁交互，使其能够在每一步获取丰富的语义信息。相反，单个查询仅向 LLM 提供初始环境设置，限制了其基于不断变化的信息提出目标的能力。这会导致生成相同的最终结果，凸显了单查询方法的灵活性有限。附录 A.1.1 中的图 9 提供了对降低成本的进一步讨论。

5 结论

在本文中，为了解决网约车系统中的动态调度问题，我们提出了一种将 LLM 和优化器结合在动态分层系统中的混合方法，其中高层模型中 LLM 生成的目标用作低层路由优化器的指导启发式。高层目标在闭环演化过程中迭代完善，并在各个模拟时期评估性能。和声搜索算法自适应地探索 LLM 提示词空间，以随着时间的推移提高客观质量。在曼哈顿市中心和芝加哥市中心出租车数据集上进行的实验证明了我们方法的有效性，与最先进的基线相比平均提高了 16% 。附录 A 中提供了其他结果、未来的工作和支持材料。源代码可以在：<https://github.com/yizhangle/llm-guided-mod-optimization> 中找到。

参考文献

- [1] Isaac Oyeyemi Olayode, Alessandro Severino, Frimpong Justice Alex, El-zbieta Macioszek 和 Lagouge Kwanda Tartibu. 网约车服务对公共道路交通影响评价的系统综述交通研究跨学科视角, 22:100943, 2023。
- [2] 安德里亚·西蒙内托、朱利安·蒙泰尔和克劳迪奥·甘贝拉。通过线性分配问题实现实时城市规模的乘车共享。交通研究 C 部分: 新兴技术, 101:208-232, 2019 年。
- [3] 菲利克斯·魏丁格、西蒙·阿尔宾斯基和尼尔斯·博伊森。自由浮动汽车共享的供需匹配: 论优化的价值。欧洲运筹学杂志, 308(3):1380-1395, 2023。
- [4] 周明, 金家瑞, 张伟南, 秦志伟, 焦艳, 王晨曦, 吴国斌, 余勇, 叶杰平。通过订单车辆分配匹配进行订单调度的多智能体强化学习。第 28 届 ACM 国际信息和知识管理会议论文集, 第 2645-2653 页, 2019 年。
- [5] 刘阳, 吴凡友, 吕成, 李申, 叶杰平, 曲晓波。深度调度: 在线约车平台车辆调度的深度强化学习方法。运输研究 E 部分: 物流与运输评论, 161:102694, 2022。
- [6] 徐哲, 赵千川, 李健, 王浩, 张伟南, 于勇。按需乘车平台中的大规模订单调度: 一种学习和规划方法。第 24 届 ACM SIGKDD 国际知识发现与数据挖掘会议记录, 第 905-913 页。ACM, 2018。
- [7] Soheil Sadeghi Eshkevari, 唐晓成, 秦志伟, 梅金瀚, 张成, 孟干英, 徐佳。野外强化学习: 在网约车市场中部署的可扩展 RL 调度算法。第 28 届 ACM SIGKDD 知识发现和数据挖掘会议论文集, 第 5033-5041 页, 2022 年。
- [8] John Holler, Risto Vuorio, 秦志伟, 唐晓成, 焦彦, 金天成, Satinder Singh, 王晨曦和叶杰平。多驾驶员车辆调度和重新定位问题的深度强化学习。2019 年 IEEE 国际数据挖掘会议记录 (ICDM), 第 1090-1095 页, 2019 年。
- [9] 王兆东, 秦志伟, 唐晓成, 叶杰平, 朱宏图。具有在线乘车订单调度知识转移的深度强化学习。2018 年 IEEE 国际数据挖掘会议记录 (ICDM), 第 617-626 页, 2018 年。
- [10] 岳新朗, 刘怡然, 施方舟, 罗思红, 陈忠, 卢敏, 徐哲。一种基于端到端强化学习的网约车微视图订单调度方法。第 33 届 ACM 国际信息和知识管理会议记录, 第 5054-5061 页, 2024 年。
- [11] 海科·霍普、托比亚斯·恩德斯、昆汀·卡帕特和马克西米利安·希弗。用于自主按需出行系统的多智能体深度强化学习的全球奖励。载于 Alessandro Abate, Mark Cannon, Kostas Margellos 和 Antonis Papachristodoulou, 编辑, 《第六届动态控制学习年度会议论文集》, 《机器学习研究论文集》第 242 卷, 第 260-272 页。PMLR, 2024 年 7 月。
- [12] 亚历山大·瓦拉尔和哈维尔·阿隆索-莫拉。优化车辆分布和车队规模, 实现按需共享出行。IEEE 机器人与自动化国际会议记录 (ICRA), 第 3853-3859 页, 2018 年。
- [13] 冯一丁, 拉德·尼亚扎德, 阿明·萨贝里。两阶段随机匹配与打车应用。2021 年 ACM-SIAM 离散算法研讨会论文集 (SODA), 第 2862-2877 页。暹罗, 2021。

[14] 比利·金和威尔·马。在线二分匹配与建议：两阶段模型的严格鲁棒性 - 一致性权衡。神经信息处理系统的进展, 35:14555-14567, 2022。

[15] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli 和 Alhussein Fawzi。使用大型语言模型进行程序搜索的数学发现。《自然》, 625(7995):468-475, 2024 年 1 月。

[16] 刘飞, 佟夏良, 袁明轩, 林曦, 罗福, 王振坤, 陆志超, 张庆福。启发式的演变: 使用大型语言模型进行高效的自动算法设计。Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett 和 Felix Berkenkamp, 编辑, 《第41届国际机器学习会议论文集》, 《机器学习研究论文集》第235卷, 第32201-32223页。PMLR, 2024 年 7 月 21 日至 27 日。

[17] 肖紫阳, 张东翔, 吴杨军, 徐丽琳, 王媛, 韩雄伟, 付晓金, 钟涛, 曾佳, 宋明丽, 等。专家链: 当 llms 遇到复杂的运筹学问题时。2023 年第十二届学习表征国际会议。

[18] 刘阳, 吴凡友, 刘志远, 王凯, 王飞跃, 曲小倩。语言模型可以用于现实世界的城市配送路线优化吗? 创新, 4(6):100453, 2023。

[19] 杨成润, 王学智, 陆一峰, 刘汉晓, Quoc V. Le, Denny Zhou, 陈新云。作为优化器的大型语言模型。arXiv 预印本 arXiv:2309.03409, 2023。

[20] Mahima Dubey, Vijay Kumar, Manjit Kaur 和 Thanh-Phong Dao。和声搜索算法的系统综述: 理论、文献和应用。工程数学问题, 2021(1):5594267, 2021。

[21] 秦峰, 阿兹兰·莫哈末·扎因, 周凯清。和声搜索算法和相关变体: 系统回顾。群体和进化计算, 74:101126, 2022。

[22] 李敏, 秦志伟, 焦岩, 杨耀东, 王军, 王晨曦, 吴国斌, 叶杰平。通过平均场多智能体强化学习实现高效的拼车订单调度。万维网会议记录 (WWW), 第983-994页, 2019年。

[23] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder 和 Martin Takác。强化学习解决车辆路径问题。神经信息处理系统的进展, 2018 年 31 月。

[24] 费德里科·罗西, 里克·张, 尤瑟夫·辛迪和马可·帕沃内。在拥挤的交通网络中路由自动驾驶车辆: 结构属性和协调算法。自主机器人, 42:1427-1442, 2018。

[25] 塔斯尼姆·艾哈迈德和萨利穆尔·乔杜里。Lm4opt: 揭示大型语言模型在制定数学优化问题方面的潜力。信息: 信息系统和运筹学, 62(4):559-572, 2024。

[27] Ali AhmadiTeshnizi, 高文治, Madeleine Udell。Optimus: 使用 (mi)lp 求解器和大型语言模型进行可扩展优化建模。arXiv 预印本 arXiv:2402.10172, 2024。arXiv:2402.10172。

[28] 刘飞, 林曦, 王振坤, 张庆福, 佟夏良, 袁明轩。具有跨问题零样本泛化的路由问题的多任务学习。第30届ACM SIGKDD 知识发现和数据挖掘会议 (KDD) 的会议记录, 第1898-1908页, 2024 年。

[29] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever 和 D. Amodei. 语言模型是小样本学习者. 神经信息处理系统进展 (NeurIPS), 第 1877-1901 页, 2020 年。

[30] 刘胜才, 陈财顺, 曲兴华, 唐克, 王耀顺. 作为进化优化器的大型语言模型. 2024 年 IEEE 进化计算大会 (CEC) 会议记录, 第 1-8 页. IEEE, 2024.

[31] Shyam Sundar Kannan, Vishnunandan L. N. Venkatesh 和 Byung-Cheol Min. Smart-llm: 使用大语言模型的智能多智能体机器人任务规划. 2024 年 IEEE/RSJ 国际智能机器人与系统会议 (IROS) 论文集. IEEE, 2024.

[32] 周嘉楠, 曹志光, 吴耀鑫, 宋文, 马一宁, 张杰, 徐驰. Mvmoe: 混合专家的多任务车辆路径求解器. arXiv 预印本 arXiv:2405.01029, 2024. <https://arxiv.org/abs/2405.01029>.

[33] 孟斯林. Llm-a*: 大型语言模型增强了路径规划的增量启发式搜索. 硕士论文, 加州大学洛杉矶分校, 2025 年。

[34] 何宇, 刘静. 深入了解大型语言模型和进化算法的自动优化. arXiv 预印本 arXiv:2410.20848, 2024.

[35] 黄金, 李新宇, 高亮, 刘启浩, 滕跃. 通过具有群体自进化的大型语言模型进行自动编程, 解决动态作业车间调度问题. arXiv 预印本 arXiv:2410.22657, 2024.

[36] Mahdi Mostajabdaveh, Timothy T. Yu, Samarendra Chandan Bindu Dash, Rindranirina Ramamonjison, Jabo Serge Byusa, Giuseppe Carenini, Zirui Zhou 和张勇. 使用 Orqa 评估运筹学领域的 llm 推理. arXiv 预印本 arXiv:2412.17874, 2024. <https://arxiv.org/abs/2412.17874>.

[37] DeepSeek-AI. Deepseek-llm: 以长期主义扩展开源语言模型. arXiv 预印本 arXiv:2401.02954, 2024.

[38] DeepSeek-AI. Deepseek-r1: 通过强化学习激励 LLMs 的推理能力, 2025 年。

[39] 纽约市出租车和豪华轿车委员会. Tlc 行程记录数据. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.

[40] 芝加哥数据门户. 出租车出行. https://data.cityofchicago.org/Transportation/出租车旅行-2013-2023-/wrvz-psew/about_data.

[41] DeepSeek. Deepseek-r1-distill-qwen-32b. <https://huggingface.co/deepseek-ai/DeepSeek-R1-蒸馏-Qwen-32B>, 2025.

[42] Gurobi 优化, 合并. Gurobi 优化器参考手册. <https://docs.gurobi.com/项目/优化器/en/current/index.html>, 2025.

[43] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lüken, Johannes Rummel, Peter Wagner 和 Evamarie Wießner. 使用相扑的微观交通模拟. <https://elib.dlr.de/124092/>, <https://sumo.dlr.de/docs/index.html>, 2018.

[44] 欧阳龙, 吴杰弗里, 江旭, 迪奥戈·阿尔梅达, 卡罗尔·温赖特, 帕梅拉·米什金, 张冲, 桑迪尼·阿加瓦尔, 卡塔琳娜·斯拉马, 亚历克斯·雷, 等. 训练语言模型遵循人类反馈的指令. 神经信息处理系统的进展, 35:27730-27744, 2022 年。

[45] 阿尔贝托·本波拉德和曼弗雷德·莫拉里. 集成逻辑、动力学和约束的系统控制. 自动, 35(3):407-427, 1999.

NeurIPS 论文检查清单

1. 研究主张

问题：摘要和引言中提出的主要主张是否准确反映了论文的贡献和范围？

答案：[是]

理由：摘要和引言准确地反映了论文的核心贡献和范围。他们清楚地阐述了主要的方法创新：将 LLM 与数学优化相结合的混合框架，用于网约车平台的动态决策。摘要和介绍适当地强调了现有 RL 和分解优化方法的局限性。他们还概述了关键组成部分，例如该方法的免训练性质、基于提示词的高层目标生成以及通过和声搜索的闭环细化机制。这些贡献在主论文（第 3.1 节和第 3.2 节以及附录算法 A.4）中得到了深入的阐述，并通过对现实世界数据集（纽约和芝加哥出租车数据）的实验（第 4 节，附录 A.1.1）进行了评估。结果证实了性能改进和语义引导优化的主张，使摘要和引言成为对论文贡献的忠实总结。

指南：

- 答案 NA 表示摘要和引言不包括论文中提出的主张。
- 摘要和 / 或引言应清楚地说明所提出的主张，包括论文中所做的贡献以及重要的假设和限制。审稿人不会很好地理解这个问题的“否”或“不适用”答案。
- 所提出的主张应与理论和实验结果相匹配，并反映结果可在多大程度上推广到其他设置。
- 只要论文明确没有实现这些目标，就可以将理想目标作为动机。

2. 局限性

问题：本文是否讨论了作者所做工作的局限性？

答案：[是]

理由：附录 A.1.2 讨论了一些局限性并概述了未来工作的方向，反映了对当前研究边界的清晰理解。具体来说，附录 A.1.2 承认模拟保真度的局限性，并指出当前的实验没有纳入细粒度的现实世界交通动态，例如拥堵模式、信号定时或道路中断。为了解决这个问题，我们建议集成 SUMO 等微观交通模拟器，以在实际操作条件下实现更高保真度的评估。此外，我们认识到 LLM 在当前框架中的静态角色：它没有经过微调，仅通过提示词级反馈进行调整。因此，我们建议未来的增强涉及强化学习技术，以实现 LLM 和优化器之间的共同适应，将优化器视为更有效地微调 LLM 的批评者。最后，我们确定了量子计算克服当前可扩展性和解决方案效率限制的潜力。我们设想将量子增强求解器集成到框架中，以加速数据驱动推理（通过 LLM）和组合优化，从而实现更紧密的交互和实时部署。

指南：

- 答案 NA 表示该论文没有限制，而答案 No 则表示该论文有限制，但这些限制并未在论文中讨论。
- 鼓励作者在论文中创建单独的“限制”部分。
- 论文应指出任何强有力的假设以及结果对于违反这些假设的鲁棒性如何（例如、独立假设、无噪声设置、模型良好规范、仅在局部保持的渐近近似）。作者

应反思这些假设在实践中可能如何被违反以及会产生什么影响。 •
 如果该方法仅在少数数据集或几次运行中进行了测试，那么作者应反思所提出的主张的范围， e . g。一般来说，实证结果通常取决于隐含的假设，这些假设应该被阐明。 •
 作者应该反思影响该方法性能的因素。例如，当图像分辨率较低或在低光照条件下拍摄图像时，面部识别算法可能表现不佳。或者，语音转文本系统可能无法可靠地用于为在线讲座提供隐藏式字幕，因为它无法处理技术术语。 •
 作者应该讨论所提出算法的计算效率以及它们如何随数据集大小进行扩展。 •
 如果适用，作者应讨论其解决隐私和公平问题的方法可能存在的局限性。 •
 虽然作者可能担心审稿人可能会利用对限制的完全诚实作为拒绝的理由，但更糟糕的结果可能是审稿人发现论文中未承认的限制。作者应该运用他们的最佳判断，并认识到有利于透明度的个人行动在制定维护社区完整性的规范方面发挥着重要作用。将特别指示审稿人不要惩罚有关限制的诚实行为。 3 理论假设与证明

问题：对于每个理论结果，论文是否提供了全套假设和完整（且正确）的证明？答案：[是]

理由：本文包含了网约车问题的正式混合整数逻辑模型表述，附录 A . 6 对此进行了介绍和讨论。还详细介绍了逻辑约束到等效混合整数线性约束的转换，并在附录 A . 7 中提供了随附的证明以确保正确性。虽然主要论文重点关注高层分配和低层路由问题的层次分解，但作为 L L M 引导优化循环的一部分，它清楚地说明了建模假设和系统级结构。此外，通过和声搜索进行模块化分解和基于提示词的细化背后的理论基础得到了附录中形式建模和完整推导的支持。指南：

- 答案 N A 表示该论文不包含理论结果。 •
- 论文中的所有定理、公式和证明都应编号并交叉引用。 •
- 所有假设都应在任何定理的陈述中清楚地陈述或引用。 •
- 证明可以出现在主论文或补充材料中，但如果出现在补充材料中，则鼓励作者提供简短的证明草图以提供直观感受。 •
- 相反，论文核心部分提供的任何非正式证明都应辅以附录或补充材料中提供的正式证明。
- 应正确引用证明所依赖的定理和引理。 4 . 实验结果重现性

问：论文是否充分披露了重现论文主要实验结果所需的所有信息，并达到影响论文主要主张和 / 或结论的程度（无论是否提供代码和数据）？答案：[是] 理由：第 4 . 1 节介绍了实验设置、评估指标和实施细节。实验中使用的纽约和芝加哥出租车数据集如附录 A . 3 所示。表 2 和图 5 提供了纽约数据集（主要论文）与基线方法的基准比较，表 3 和图 6 提供了芝加哥数据集的基准比较。

进行消融研究是为了评估系统的几个关键方面。表 4 和图 4 分析了场景提示词组成的影响。表 5 详细介绍了动态环境反馈机制的影响。附录中提供的其他分析包括：多查询和单查询设置的迭代成本变化（图 9）；进化超参数研究（图 10 和 11）；LLM 温度敏感性的调查（图 12）；以及 LLM 引起的错误率分析（表 6）。为了保持贡献的完整性并确保正确的归属，<https://github.com/yizhang/llm-guided-mod-optimization> 中提供了代码以支持未来的研究。指南：

- 答案NA 表示论文不包含实验。
- 如果论文包含实验，审稿人不会很好地理解此问题的“否”答案：无论是否提供代码和数据，使论文具有可重复性非常重要。
- 如果贡献的是数据集和/或模型，作者应描述为使其结果可重复或可验证而采取的步骤。
- 根据贡献，可通过多种方式实现再现性。例如，如果贡献是一种新颖的架构，则完全描述该架构可能就足够了，或者如果贡献是一个特定的模型和实证评估，则可能有必要让其他人能够使用相同的数据集复制该模型，或者提供对该模型的访问。一般来说，发布代码和数据通常是实现这一目标的一种好方法，但也可以通过详细说明如何复制结果、访问托管模型（例如，在大型语言模型的情况下）、发布模型检查点或适合所执行的研究的其他方式来提供可重复性。
- 虽然NeurIPS 不要求发布代码，但会议确实要求所有提交内容提供一些合理的再现性途径，这可能取决于贡献的性质。例如（a）如果贡献主要是一个新算法，论文应该明确如何重现该算法。（b）如果贡献主要是一个新的模型架构，论文应该清晰、完整地描述该架构。（c）如果贡献的是一个新模型（例如，一个大型语言模型），那么应该有一种方法来访问该模型以重现结果，或者一种重现模型的方法（例如，带有开源数据集或如何构建数据集的说明）。（d）我们认识到，在某些情况下，可重复性可能很棘手，在这种情况下，欢迎作者描述他们提供可重复性的特定方式。在闭源模型的情况下，模型的访问可能会受到某种方式的限制（例如，仅限注册用户），但其他研究人员应该有可能有某种途径来复制或验证结果。
- 5 . 开放数据和代码的访问

问题：论文是否提供对数据和代码的开放访问，并提供足够的说明来忠实地再现主要实验结果，如补充材料中所述？答案：[是] 理由：我们的代码库现已公开，可以在<https://github.com/yizhang/llm-guided-mod-optimization> 中找到。指南：

- 答案NA 表示论文不包含需要代码的实验。
- 请参阅NeurIPS 代码和数据提交指南(<https://nips.cc/public/guides/CodeSubmissionPolicy>) 了解更多详情。
- 虽然我们鼓励发布代码和数据，但我们知道这可能是不可能的，因此“否”是可以接受的答案。论文不能仅仅因为不符合而被拒绝

包括代码，除非这是贡献的核心（例如，用于新的开源基准）。
 说明应包含运行以重现结果所需的确切命令和环境。有关更多详细信息，请参阅 NeurIPS 代码和数据提交指南 (<https://nips.cc/public/guides/CodeSubmissionPolicy>)。
 作者应提供有关数据访问和准备的说明，包括如何访问原始数据、预处理数据、中间数据和生成数据等。
 作者应提供脚本来重现新提出的方法和基线的所有实验结果。如果只有一部分实验是可重复的，他们应该说明脚本中省略了哪些实验以及原因。
 在提交时，为了保持匿名，作者应发布匿名版本（如果适用）。
 建议在补充材料（附加到论文中）中提供尽可能多的信息，但允许包含数据和代码的 URL。

6. 实验设置 / 细节

问题：论文是否指定了理解结果所需的所有训练和测试细节（例如、数据分割、超参数、如何选择它们、优化器类型等）？答案：[是]
 理由：虽然我们的工作不涉及传统意义上的机器学习模型训练，但它将完善的预训练 LLM 与数学优化框架集成在一起。因此，没有需要报告的训练 / 测试数据分割或模型训练程序。然而，为了确保结果的清晰度，我们提供：（1）第 4.1 节实施细节中涉及我们的方法的所有组件的完整实施细节；（2）纽约和芝加哥出租车数据集以及生成九个测试场景的方法在附录 A.3 中描述；（3）为了发展 LLM 的提示词启发式，我们利用和声搜索算法。该算法的关键超参数（例如、HMC R 和 PAR）通过附录 A.1.1 中图 10 和 11 所示的敏感性研究进行分析。这些细节共同提供了理解结果所需的实验设置的完整图片。
 指南：

• 答案 NA 表示论文不包含实验。
 实验设置应在论文的核心部分详细介绍，以便理解结果并理解结果。
 完整的详细信息可以在代码中、附录中或作为补充材料提供。

7. 实验统计显著性

问题：论文是否报告了适当且正确定义的误差线或有关实验统计显著性的其他适当信息？
 答案：[是] 理由：我们的研究不涉及训练数据驱动模型，而是评估将基于 LLM 的提示词生成与优化求解器集成的混合框架的性能和稳定性。为了证明统计显著性和变异性：（1）我们在两个不同的现实世界出租车数据集（纽约和芝加哥）上对我们的方法进行基准测试，提供对不同行程分布的适应性的证据。（2）我们评估提示词合成对 LLM 推理稳定性的影响。具体来说，表 4 报告了四种场景和三种提示词变体的平均乘客等待时间，显示了每种设置 3 次独立运行的平均值和标准差。这些误差线反映了由于随机性和固有的不确定性而导致的 LLM 输出的变化。（3）我们通过错误率分析研究 LLM 输出的有效性（附录 A.1.1 中的表 6），显示 Gurobi 求解器在提示词类型和场景中对 LLM 生成的目标的接受率。每个配置都会通过三次独立运行进行评估，并报告平均错误率。

所有报告的统计数据都阐明了变异性的来源（LLM 生成随机性）和所使用的方法（多次独立运行的经验评估）。明确提供了标准差，并在正文中交叉引用了表格 / 图形。

指南：

- 答案NA 表示论文不包含实验。
- 如果结果附有误差线、置信区间或统计显著性检验（至少对于支持论文主要主张的实验），作者应回答“是”。
- 应明确说明误差条捕获的变异因素（例如，训练 / 测试分割、初始化、某些参数的随机绘制或给定实验条件下的总体运行）。
- 应解释计算误差条的方法（封闭式公式、调用库函数、引导程序等）。
- 应给出所做的假设（例如，正态分布误差）。
- 应清楚误差条是标准差还是平均值的标准误差。
- 报告 $1 - \sigma$ 误差线是可以的，但应该说明这一点。如果错误正态性假设未得到验证，作者最好报告 $2 - \sigma$ 误差线，而不是声明其 CI 为 96%。
- 对于非对称分布，作者应注意不要在表格或图中显示对称误差条，否则会产生超出范围的结果（e.g. 负误差率）。
- 如果在表格或图中报告了误差线，作者应在文本中解释其计算方法，并在文本中引用相应的图形或表格。

8. 实验计算资源

问题：对于每个实验，论文是否提供了重现实验所需的计算机资源（计算工作者类型、内存、执行时间）的足够信息？

答案：[是]

理由：本文提供了有关第 4.1 节中使用的计算资源的足够详细信息。具体来说，所有基于优化器的方法均在配备第 13 代 Intel Core i9-13900KF CPU（32 核，高达 5.80 GHz）和 32 GB RAM 的本地计算机上执行。对于基于 LLM 的方法，通过 Hugging Face 平台 API 访问 DeepSeek-R1-Distill-Qwen-32B 模型，并报告相关配置参数（例如，温度 = 0.9）。每个基于 LLM 的方法在每个场景中执行 3 次，并给出平均结果。然而，由于这些方法依赖于第三方 API 调用，端到端运行时受到 Hugging Face 平台上的网络延迟和服务器负载的影响，因此很难准确报告挂钟执行时间。尽管如此，迭代次数、群体规模和评估方案都得到了明确说明，确保了实验程序的可重复性。

指南：

- 答案NA 表示论文不包含实验。
- 论文应指出计算工作人员 CPU 或 GPU、内部集群或云提供商的类型，包括相关内存和存储。
- 论文应提供每次单独实验运行所需的计算量并估计总计算量。
- 论文应披露整个研究项目是否需要比论文中报告的实验更多的计算（例如，未纳入论文的初步或失败的实验）。

9. 道德准则

问题：论文中进行的研究是否在各个方面都符合 NeurIPS 道德准则 <https://neurips.cc/public/EthicsGuidelines>？

答案：[是]

理由：该研究的重点是使用既定的优化基准进行算法开发和评估。它不涉及人类参与者或涉及引起明显道德问题的应用程序。基于此，我们确认该工作完全符合 NeurIPS 道德准则。

指南：

- 答案NA 表示作者尚未审查 NeurIPS 道德准则。
- 如果作者回答“否”，他们应解释需要偏离《道德准则》的特殊情况。
- 作者应确保保持匿名（例如，如果由于其管辖范围内的法律或法规有特殊考虑）。

10. 更广泛的影响

问题：本文是否讨论了所开展工作的潜在积极社会影响和消极社会影响？

答案：[否]

理由：本文强调技术贡献，但没有明确解决更广泛的社会影响。尽管应用领域（例如交通系统）表明了潜在的积极社会效益，但并未具体阐明。同样，手稿中也没有讨论可能的负面社会后果。

指南：

- 答案“NA”意味着所做的工作不会产生社会影响。
- 如果作者回答“不适用”或“否”，他们应该解释为什么他们的工作没有社会影响或为什么论文没有解决社会影响。
- 负面社会影响的例子包括潜在的恶意或非预期使用（例如、虚假信息、生成虚假个人资料、监视）、公平性考虑（例如、部署可能做出不公平影响特定群体的决策的技术）、隐私考虑和安全考虑。
- 会议预计许多论文将是基础研究，不与特定应用程序相关，更不用说部署了。然而，如果存在任何负面应用的直接路径，作者应该指出。例如，可以合理地指出，生成模型质量的改进可用于生成虚假信息的深度伪造品。另一方面，无需指出优化神经网络的通用算法可以使人们能够更快地训练生成 Deepfakes 的模型。
- 作者应考虑当技术按预期使用且正常运行时可能出现的危害、当技术按预期使用但给出不正确结果时可能出现的危害、以及（有意或无意）滥用技术而造成的危害。
- 如果存在负面社会影响，作者还可以讨论可能的缓解策略（例如，模型的门控发布，除了攻击之外还提供防御，监控滥用的机制，监控系统如何随着时间的推移从反馈中学习的机制，提高机器学习的效率和可访问性）。

11. 保障措施

问题：本文是否描述了为负责任地发布滥用风险较高的数据或模型（例如、预训练语言模型、图像生成器或抓取的数据集）而采取的保障措施？

答案：[不适用]

理由：本文介绍了一种将 LLM 与优化技术相结合的新颖框架。优化模型、其与 LLM 的集成以及所采用的标准基准数据集不会带来很高的误用风险。因此，除了标准开源实践之外，没有必要采取额外的发布保障措施。

指南：

- 答案NA 表示该论文不存在此类风险。

具有高误用或双重用途风险的已发布模型应在发布时采取必要的防护措施，以允许模型的受控使用，例如要求用户遵守使用指南或访问模型的限制或实施安全过滤器。从互联网上抓取的数据集可能会带来安全风险。作者应该描述他们如何避免发布不安全的图像。我们认识到提供有效的保障措施具有挑战性，许多论文并不要求这样做，但我们鼓励作者考虑到这一点并尽最大努力。 12. 现有资产的许可证

问题：论文中使用的资产（例如，代码、数据、模型）的创建者或原始所有者是否得到了适当的认可，并且是否明确提及并适当尊重了许可和使用条款？答案：[是]
理由：本文适当引用并注明了所有外部资产，包括基线方法（例如，默认 + RL - Seq、Full RL、FunSearch、EoH）和数据集（纽约出租车、芝加哥出租车）。详情如下：

• 默认 + RL - Seq：此方法采用深度强化学习进行低层测序。我们在手稿中引用了相关出版物，并利用了 GitHub 上公开的实现。尽管该存储库没有明确指定许可证，但我们已联系该存储库的作者以澄清使用条款。
<https://github.com/higgsfield/np-hard-deep-reinforcement-learning/> 树/主
• Full RL：该方法应用强化学习来解决整个问题。该实现可在 GitHub 上公开访问。尽管该存储库没有指定许可证，但我们手稿中引用了相关出版物，并联系了 GitHub 存储库的作者以澄清使用权限。我们已通过电子邮件收到确认，表明我们允许使用他们的代码。
<https://github.com/callmespring/MDPOD/tree/main>。
• EoH：该代码可在 MIT 许可下在 GitHub 上公开获取。
<https://github.com/FeiLiu36/EoH/blob/main/LICENSE>。
• FunSearch：我们采用 EoH 基线方法中的 FunSearch 实现，该方法是公开可用的，并根据 Apache 2.0 许可证发布。
<https://github.com/FeiLiu36/EoH/blob/main/baseline/funsearch/> 许可证。
纽约出租车数据集：该数据集由 NYC 出租车和豪华轿车委员会公开提供并维护。它可以通过 NYC 开放数据门户访问，并根据 NYC 开放数据使用条款提供，该条款通常允许出于商业和非商业目的使用、修改和重新分发，并注明归属。
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
<https://opendata.cityofnewyork.us/overview/#termsfuse>。
芝加哥出租车数据集：该数据集可通过芝加哥市数据门户公开获取。虽然没有列出具体许可证，但它旨在供公众访问和使用。该市的数据免责声明概述了一般条款，通常允许使用、修改和重新分发并具有适当的归属。
https://data.cityofchicago.org/Transportation/Taxi-Trips-2013-2023-/wrvz-psew/about_data
https://www.chicago.gov/city/en/narr/foia/data_disclaimer.html。
DeepSeek - R1 - Distill - Qwen - 32B：我们通过 Hugging Face 平台 API 利用这个公开的大型语言模型进行快速优化和基于 LLM 的决策。该模型是根据 MIT 许可证发布的。DeepSeek - R1 - Distill - Qwen - 32B 源自 Qwen - 2.5 系列（最初根据 Apache 2.0 许可证授权），并由 DeepSeek - R1 对 800k 精选样本进行进一步微调。我们在手稿中引用了官方模型存储库，并遵守所有许可条款。
<https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>

指南：

- 答案N A 表示本文不使用现有资产。
- 作者应引用生成代码包或数据集的原始论文。
- 作者应说明所使用的资产版本，并在可能的情况下包含 URL。
- 每项资产均应包含许可证名称（例如，CC - BY 4.0）。
- 对于从特定来源（例如、网站）抓取的数据，应提供该来源的版权和服务条款。
- 如果发布资产，则应提供包中的许可证、版权信息和使用条款。对于流行的数据集，`paperswithcode.com/datasets` 已为某些数据集策划了许可证。他们的许可指南可以帮助确定数据集的许可。
- 对于重新打包的现有数据集，应提供原始许可证和派生资产的许可证（如果已更改）。
- 如果无法在线获取此信息，我们鼓励作者联系该资产的创建者。 13 . 新资产

问题：本文中引入的新资产是否有详细记录，并且文档是否与资产一起提供？答案：[是] 理由：我们的代码库以及全面的文档现已在 `https://github.com/yizhang/llm-guide-d-mod-optimization` 指南中提供：

- 答案N A 表示该论文不发布新资产。
- 研究人员应通过结构化模板传达数据集 / 代码 / 模型的详细信息，作为其提交内容的一部分。这包括有关培训、许可、限制等的详细信息。
- 文件应讨论是否以及如何获得资产使用者的同意。
- 提交时，请记住对您的资产进行匿名化（如果适用）。您可以创建匿名 URL 或包含匿名 zip 文件。 14 . 众包和人类受试者研究

问题：对于以人类为对象的众包实验和研究，论文是否包含向参与者提供的说明全文和屏幕截图（如果适用）以及有关补偿的详细信息（如果有）？答案：[不适用] 理由：该研究不涉及众包实验或人类受试者研究。指南：

- 答案N A 表示该论文不涉及众包或人类受试者研究。
- 在补充材料中包含此信息很好，但如果论文的主要贡献涉及人类受试者，则应在主论文中包含尽可能多的细节。
- 根据 NeurIPS 道德准则，参与数据收集、管理或其他劳动的工人应至少获得数据收集者所在国家 / 地区的最低工资。 15 . 人体受试者研究的机构审查委员会（IRB）批准或同等批准
- 问题：论文是否描述了研究参与者产生的潜在风险，是否向受试者披露了这些风险，以及是否获得了机构审查委员会（IRB）批准（或根据您所在国家或机构的要求进行的同等批准 / 审查）？

答案：[不适用] 理由：该研究不涉及人类受试者，因此 I R B 批准不适用。指南：

- 答案N A 表示该论文不涉及众包或人类受试者研究。
 - 根据进行研究的国家 / 地区，任何人类受试者研究可能需要 I R B 批准（或同等文件）。如果您获得了 I R B 批准，您应该在文件中明确说明这一点。
 - 我们认识到，不同机构和地点的程序可能存在很大差异，我们希望作者遵守 NeurIPS 道德准则及其机构指南。
 - 对于初次提交的信息，请勿包含任何会破坏匿名性的信息（如果适用），例如进行审查的机构。
16. LLM用途声明

问题：如果 L L M 是本研究核心方法的重要、原创或非标准组成部分，论文是否描述了 L L M 的用法？请注意，如果 L L M 仅用于写作、编辑或格式化目的，且不影响研究的核心方法、科学严谨性或原创性，则不需要声明。答案：[是]

理由：我们论文的核心贡献是将 L L M

集成到网约车平台的在线优化框架中。L L M 不仅用于写作或格式化，而且在方法论中发挥着核心、原创的作用。具体来说，我们提出的框架采用 L L M 作为元优化器，生成高层语义启发式方法来指导负责在操作约束下进行实时决策的较低层数学优化器。该方法通过消除对训练数据的需求并实现自适应目标制定，解决了传统强化学习和分解优化方法的关键局限性。

L L M 的用法在 3.2 节中详细描述，其中我们介绍了混合 L L M + 优化器框架。完整的闭环机制，包括如何使用进化反馈来完善提示词，在算法 A.4

中进行了进一步详细说明。我们还在附录 A.9

中讨论了我们的即时工程策略。这些组件展示了 L L M 的新颖且非标准的使用，直接影响我们系统的原创性、方法的严谨性和性能。指南：

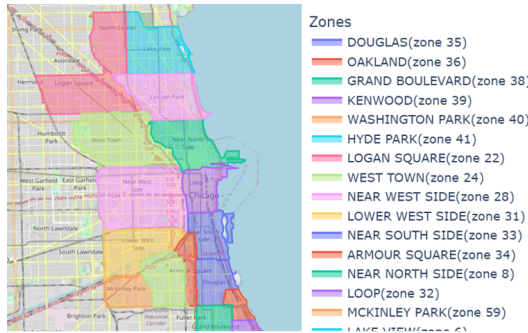
- 答案N A
- 意味着本研究中的核心方法开发不涉及 L L M 作为任何重要的、原创的或非标准的组成部分。
- 请参阅我们的 L L M 政策 (<https://neurips.cc/Conferences/2025/LLM>) 了解应该或不应该描述的内容。

A 技术附录

A.1 讨论

A.1.1 进一步研究

时空分析 为了说明不同方法中乘客等待时间的时空分布，图 7 和图 8 分别显示了曼哈顿市中心和芝加哥市中心场景 9（如表 2 和表 3 中定义）的结果，其中 200 名乘客请求在 1200 秒的时间窗口内调度，并由 100 辆出租车提供服务。为了便于对研究区域进行空间解释，图 5 和图 6 分别显示了曼哈顿市中心和扩展的芝加哥市中心的区域地图，每个研究区域分为 19 个预定义区域，用作旅客出行的出发地和目的地。



与曼哈顿的分区结构相比，芝加哥的分区（也称为社区区域）的空间范围通常更大，特别是在中央商务区之外。特别是，OD 出行模式揭示了两个城市之间的显著对比：曼哈顿的出租车出行在其区域内分布相对均衡，而芝加哥的出租车活动高度集中在几个中心区域，最突出的是 8、32 和 28 区。附录 A.3 中提供了有关 OD 分布的更多详细信息。

图 7 和图 8 分别显示了曼哈顿市中心和芝加哥市中心各区域乘客等待延误的时空分布。两个图均对应于高需求场景 9，如表 2 和表 3 中所定义。乘客等待延迟的计算公式为： $delay_p = \max(t_{pick_p} - t_{req_p}, 0)$ ，其中 t_{pick_p} 表示车辆接载时间， t_{req_p} 表示乘客请求时间。对于图 7 和图 8，延迟分别聚合为 600 秒时间 bin（例如， $t_{pick_p} \in [600s, 1200s]$ 映射到时隙 1）和 800 秒时间 bin（例如， $t_{pick_p} \in [800s, 1600s]$ 映射到时隙 1）。根据聚合的数据块，延迟在一组预定义的地理区域上进行空间平均。这种联合时空聚合捕获了局部服务效率低下的问题，同时保持了供需不平衡随时间的动态变化。热图定量地验证了我们框架的优势：在相同的输入条件下，我们的方法在 4 个时隙内完成所有乘客接送（图 7 为 4×600 秒，图 8 为 4×800 秒），而基线方法在曼哈顿设置中需要 5 到 7 个时隙，在芝加哥设置中需要 5 到 9 个时隙才能满足所有请求。随着时间的推移，未得到服务的乘客的延迟不断累积，导致后面时段段的平均等待时间显著增加。这种效果在热图中反映为最终时间箱中逐渐增强的红色阴影。由于手动客观方法（表 2 和表 3）通常会产生较高的乘客延迟，因此图 7 和图 8 中仅包含表现最佳的方法（距离 $\times$ 时间 $\times$ 利用率）以进行比较。

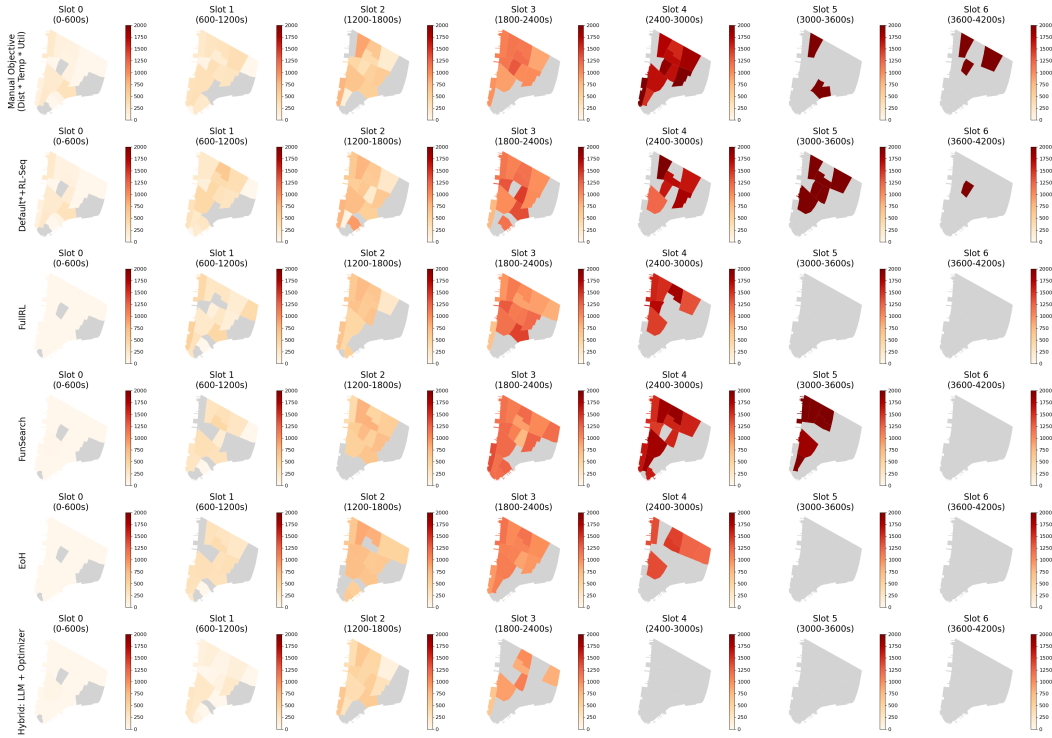


图 7：纽约出租车数据集下场景 9（P200_C100_T1200）中乘客等待时间跨优化方法的时空分布。每行代表一种方法：（1）手动复合目标（距离 $\times$ 时间 $\times$ 利用率），（2）默认 + RL-Seq，（3）完整 RL，（4）FunSearch，（5）EoH，（6）我们的 LLM 优化器方法。列显示 600 秒的时间窗口。热图颜色显示每个区域的平均等待时间。

在图 7 中，由于静态设计的目标，第一种方法（第 1 行）显示出 144、79 和 249 区的持续热点。RL 方法（第 2 - 3 行）显示时间退化，尤其是第 2 行，其中时隙 5（3000 - 3600 秒）中扩展的高延迟区域突出了 RL 在强制执行硬约束（例如，绕行限制）方面的局限性，并推广到看不见的需求模式。基于 LLM 的方法（尤其是 EoH）的性能稍好一些，可以在 5 个时隙内完成所有请求。尽管 FunSearch 是跨场景的最佳整体基线，因为它优于我们在场景 1（0.77 分钟）和场景 3（1.55 分钟）中的方法，如表 2 所示，但它的性能在高需求条件下显著下降，特别是在图 7 中捕获的场景 9 中。相比之下，我们提出的框架在 4 个时隙内完成所有请求，同时通过以下方式保持空间一致的低延迟（浅黄色）闭环 LLM - 优化器交互。该分析量化了我们的框架如何克服 RL 的数据依赖性、手动分解目标的刚性以及仅 LLM 方法的约束无意识。热图与表 2 中的场景 9 结果一致，证明语义推理（LLM）和符号基础（求解器）可以有效减轻大规模城市交通系统中的延迟累积。

图 8 中观察到了类似的模式。与曼哈顿数据集的结果不同，芝加哥场景需要更多时间来完成所有请求，这主要是由于行程分布更加不平衡以及起点 - 目的地对之间的平均行程时间更长。具体来说，第一个（第 1 排）和第二个（第 2 排）基线进场分别需要 9 个和 8 个时段来为所有乘客提供服务。完整的 RL（第 3 行）和 EoH（第 5 行）展示了性能的改进，在 5 个时隙内完成了任务，而 FunSearch（第 4 行）在 6 个时隙内表现出适度的退化。相比之下，所提出的方法仅在 4 个时隙内完成所有请求，表明对复杂城市拓扑和需求异质性的适应性增强。这凸显了将 LLM 作为数据驱动先验与数学优化集成以实现稳健、高效决策的有效性。

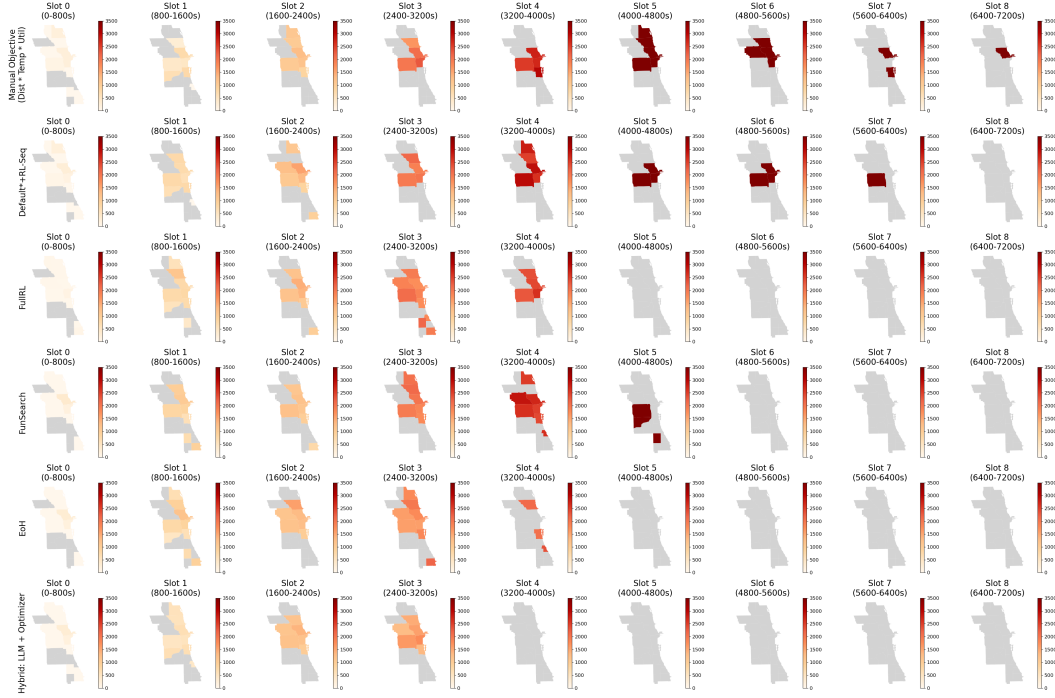


图 8：芝加哥出租车数据集下场景 9（P200_C100_T1200）中乘客等待时间跨优化方法的时空分布。每行代表一种方法：（1）手动复合目标（距离 $\times$ 时间 $\times$ 利用率），（2）默认 + RL-Seq，（3）完整 RL，（4）FunSearch，（5）EoH，（6）我们的 LLM 优化器方法。列显示 800 秒的时间窗口。热图颜色显示每个区域的平均等待时间。

查询比较图 9 显示了多查询和单查询不同迭代下的成本变化，对应于表 5 中描述的场景。在所有运行中，多查询实现了更陡峭的收敛轨迹，在 10 次迭代内将 costs 平均减少了 50%（相对于单查询），这与表 5 一致。这是 LLM 与优化器之间的闭环交互的结果。多查询框架，与单查询方法的开环配置形成对比。

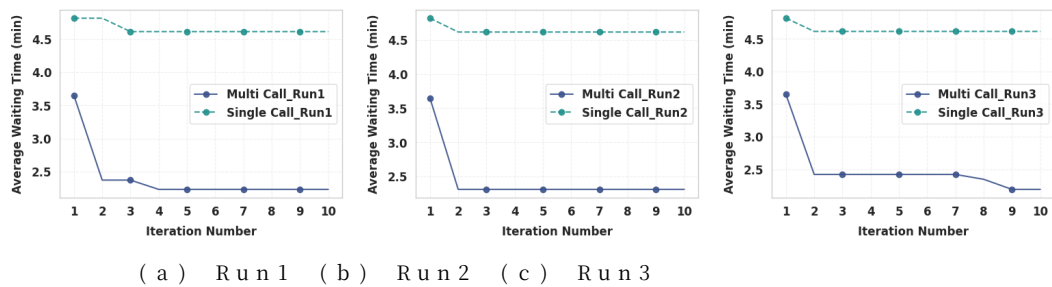


图 9：通过多次调用和单次调用方法之间的迭代，成本发生变化。

进化超参数研究 如图 10 所示，在 P100_C80_T900 的和声搜索算法中对超参数 HMCR 和 PAR 执行网格搜索。测试的 HMCR 值为 0.1、0.5 和 0.9，而测试的 PAR 值为 0.2、0.5 和 0.8。图 10a 和图 10b 描述了上次迭代中的最佳个体成本和上次迭代中整个群体的平均成本。结果表明，在高 HMCR（HMCR = 0.9）下，性能显着提高，表明可以有效利用内存中的高质量父解决方案。这

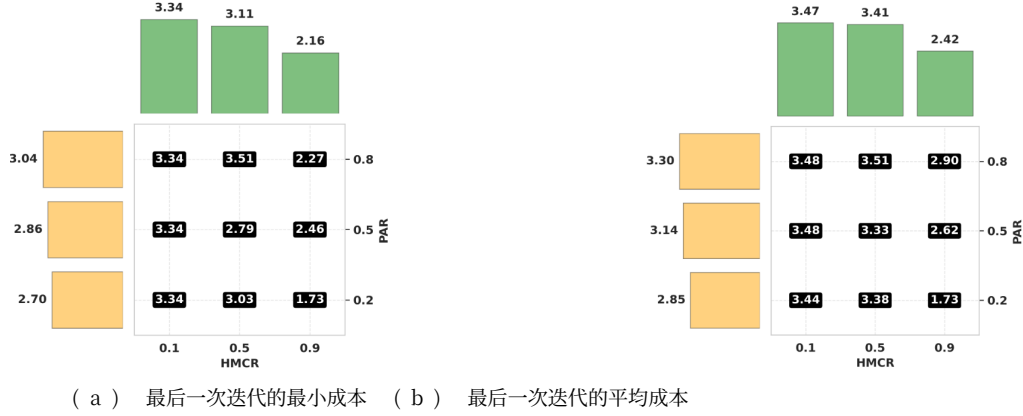


图 10：HMC R 和 P A R 的超参数敏感性分析 - 最后一次迭代的成本。

与我们的假设一致，即通过频繁的内存调用（通过算子 W2 和 W3）保留有前途的解决方案组件可以显著增强收敛特性。另一方面，虽然较低的 P A R 值（0.2）在 HMC R = 0.9 时产生最佳结果，但对于 HMC R ≤ 0.5，这种关系不再存在。因此，W2 和 W3 之间的操作员选择频率似乎更多地受到解决方案质量反馈的控制，而不是固定参数化。图 11 a、11 b 和 11 c 中观察到类似的成本降低模式，当 HMC R 较高时，成本显着降低。

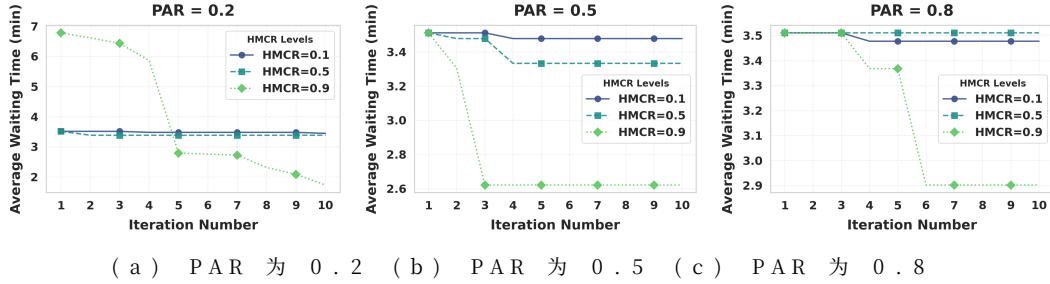


图 11：HMC R 和 P A R 的超参数敏感性分析 - 各种迭代的成本。

LLM 温度研究 我们评估了 LLM 温度 ($\tau \in \{0.1, 0.5, 0.9, 1.3\}$) 对三种情况下优化效率的影响，如图 12 所示。等高线图描绘了 10 次迭代和 4 个温度下的成本情况，揭示了极端温度 ($\tau = 0.1$ 和 $\tau = 1.3$) 始终导致更高的总体效率 costs，而 $\tau = 0.9$ 产生最低值，表明优化更有效。在所有设置中，收敛速度很快，轨迹在 2 - 3 次迭代内稳定下来。当 $\tau = 1.3$ 时，高随机性会破坏目标函数的一致性，从而生成不可行或不稳定的公式。而 $\tau = 0.1$ 过度限制多样性，导致过早收敛到次优盆地。 $\tau = 0.9$ 的中间设置在利用（利用高似然标记来遵守约束）和探索（对新颖的目标结构进行采样）之间取得了有效的平衡。这些发现强调温度是调节 LLM 优化器共生的关键超参数。

错误率研究 正如实验 4.3 中所讨论的，表 6 量化了 LLM 在四种场景和三种提示词组合中生成的目标函数的失败率，并在最后一次迭代（第 10 次迭代）时测量。对于每种场景，LLM 查询随时间窗口持续时间缩放：P50_C30_T300（每个人 1 个查询，整个 popsize 的总响应数为 5 个）到 P130_C80_T1200（每个人 4 个查询，总共 20 个响应）。错误率计算为 # 个无效目标 / 总回复数。

当仅提供 Psys Upgeo Updata 时，在所有情况下错误率都达到 100%，因为不充分的提示词会产生语义上无效的表述。用 Pmodel 替换 Pdata 将误差降低至 20%（P50）和 36.7%（P130），证明了通过结构先验进行任务接地的必要性。更远

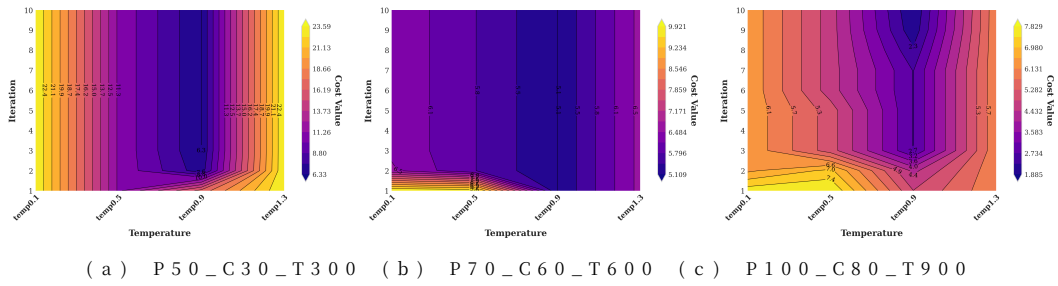


图 12：不同温度设置和迭代的成本情况。

在较小的场景（P50 - P100）中，`Restriction` 的加入将错误最小化至 0.0%，在 P130 中将错误最小化至 8.3%，突出了约束形式化的作用。这表明通过模型结构化和约束形式化进行渐进式丰富，在确保 LLM 驱动的优化管道的可行和有效输出方面发挥着关键作用。此外，对于后两种提示词组合，随着场景复杂性的增加，错误率呈现上升趋势（Pmodel 从 P50 的 20.0% 上升到 P130 的 36.7%，P50 从 P50 的 0.0% 上升到 `Restriction` 的 P130 的 8.3%），反映出 LLM 在高维空间中生成的目标不断升级的脆弱性。

表 6：不同提示词组合下 LLM 生成解的错误率分析

场景提示词成分	Run1	Run2	Run3	平均
P50_C30_T300	Psys	UPgeo	UPdata	100% 100%
100% 100.0%	Psys	UPgeo	UPmodel	0% 0% 60%
20.0% + Restriction	0%	0%	0%	0.0%
P70_C60_T600	Psys	UPgeo	UPdata	90% 100%
100% 96.7%	Psys	UPgeo	UPmodel	0% 0% 0%
0.0% + Restriction	0%	0%	0%	0.0%
P100_C80_T900	Psys	UPgeo	UPdata	100% 100%
100% 100.0%	Psys	UPgeo	UPmodel	13.3% 60%
26.7% 33.3% + Restriction	0%	0%	0%	0.0%
P130_C80_T1200	Psys	UPgeo	UPdata	100% 100% 100%
100.0% Psys	UPgeo	UPmodel	55% 25% 30%	36.7% +
Restriction	0%	15%	10%	8.3%

注：场景表示乘客数量（P）、出租车（C）和时间窗口（T）。提示词组件：系统（Psys）、几何体（Pgeo）、参数/变量格式（Pdata）、模型蓝图（Pmodel）和约束（Restriction）。+Restriction 表示 Psys UPgeo UPmodel UPrestriction。

A.1.2 未来工作

应用层可扩展性和模拟保真度我们的实验证明了使用 32B 参数 LLM 进行的有效优化，表明较小的模型在适当约束特定于领域的推理时可以实现相当的性能。未来，我们将集成微观交通模拟器（例如，SUMO [43]），以实现路网动态的高保真建模，包括拥堵、信号配时和具有空间相关性的随机旅行时间。这种集成将支持在实际操作条件下评估我们的框架，考虑到需要动态重新规划的瞬时中断（例如、事故、道路封闭）。

LLM-Optimizer 协同适应的强化学习当前框架将 LLM 视为固定代理，通过基于提示词的交互迭代地适应其目标生成行为，它不会修改模型的内部参数。然而，我们系统的结构（LLM 提出目标并由外部优化器评估它们）反映了强化学习中的行动者批评范式。这开辟了一个有前途的方向：利用优化器反馈（例如，fitness costs）直接微调 LLM，有可能以更小的规模实现高效训练

模型。通过将优化器集成为评估 LLM 提议目标的批评者，我们可以超越即时工程，进入 LLM 经过明确训练以随着时间的推移生成更有效目标的环境。鉴于之前使用 RL 有效微调 LLM 的成功 [44]，这种优化器引导的微调可以实现更高效的样本和领域自适应优化管道，进一步连接大型语言模型和传统求解器。

量子增强的分层优化所提出的框架在分层交互中利用 LLM 和经典优化求解器来解决顺序决策问题，我们预计量子计算的进步可以显著增强这种范式。尽管量子计算仍处于早期阶段，但它在比经典计算机更有效地解决组合和离散优化问题方面表现出了强大的潜力。随着量子硬件的成熟，它可以加速 LLM 优化器交互的双方，为 LLM 引导的启发式方法提供更快的推理，并从量子增强优化器提供更高效率的解决方案。这将使推理和解决组件之间的集成更加紧密，交互更加频繁，为我们的框架的实时实现及其在动态决策环境中的部署打开了大门。

A.2 基线方法

手动设计的目标：我们通过结合不同的调度优先级（例如、距离、时间和利用率）来构建一组手工设计的目标基线，这受到之前两项工作的启发：[2]，它提出了基于距离的建模分配（用作我们的仅距离基线），以及 [24]，它引入了结合时间和利用率方面的 MILP 模型（反映在我们的时间 $\times$ 利用率基线中）。在此基础上，我们还包括两个额外的组合：距离 $\times$ 利用率、距离 $\times$ 时间 $\times$ 利用率。这些基于规则的目标形成了手动目标基线，用于评估各种调度场景的性能。二级路由采用公式 1 的优化模型。

Default + RL-Seq：顾名思义，Default + RL-Seq 使用默认目标 $Distance \times Temporal \times Utilization$ 进行第一层分配。然而，对于二级路由，它采用 [23] 中提出的基于 RL 的方法。更具体地说，我们训练了一个策略模型，在一级分配优化后随机预测接载乘客的顺序，从而最大限度地减少总旅行时间。在我们的实施中，对于乘客和出租车，我们使用往返所有可能地点的旅行时间作为策略模型的输入特征。我们使用与 [23] 类似的架构，但禁用瞥见来增强学习。在推理过程中，使用基于搜索的策略来决定最终的取货顺序。

FullRL：FullRL 是在线订单调度策略的实现 [6]，其中基于优势函数估计 $A\pi(i, j) = \gamma \Delta t_j V(s'_{ij}) - V(s_i) + R\gamma(j)$ 在滚动范围内做出决策，其中 γ 是折扣因子， Δt_j 是匹配的司机 i 和订单 j 的取货时间。奖励 $R\gamma(j)$ 是调度司机 i 去订单 j 的奖励。为了与本文中的问题设置同步，我们修改奖励以反映订单 j 的等待时间，而不是使用默认值 1。因此，目标从最大化服务率（如原始论文中）转变为最小化总等待时间。驾驶员 i 的当前状态表示为二维向量（位置和时间）。驾驶员当前状态的价值函数会对优势函数 $(-V(s_i))$ 产生负面影响。将订单分配给目的地位于更有价值区域的驾驶员的动作会产生更高的优势，表示为 $V(s'_{ij})$ 。最后，每个时间戳的调度决策基于优势函数估计。

FunSearch：FunSearch [15] 也利用 LLM 来产生给定问题的方法。在测试过程中，我们进行了一些调整以适应出租车乘客分配问题。更具体地说，扩展通用提示词，我们提供了有关方法输入和输出格式的信息，并更明确地定义了优化目的。此外，我们还调整了超参数，以鼓励在寻找最终方法时进行更多探索。对于每个场景，我们在报告最终结果之前采样了 20 种方法。

EOH EOH [16] 是一个结合了 LLM 和进化计算来自自动化启发式设计的框架。它利用五种提示词驱动策略，并将启发式想法表达为自然语言“想法”。这些想法随后被 LLM 转换为可执行代码，这有助于改进和完善原始启发式方法。在我们的实验中，我们使用约束功能提示词来评估 EOH 的出租车乘客分配问题。目标是尽量减少乘客等待时间

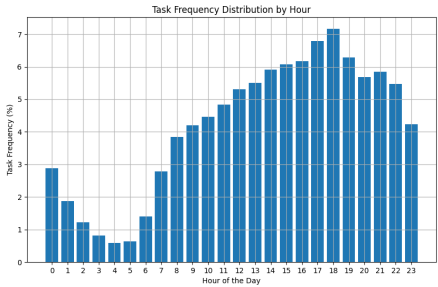
确保出租车利用的平衡和高效。我们对 9 个场景进行了评估，其中有 10 次进化迭代，种群规模为 5。

A.3 数据集描述与测试场景生成

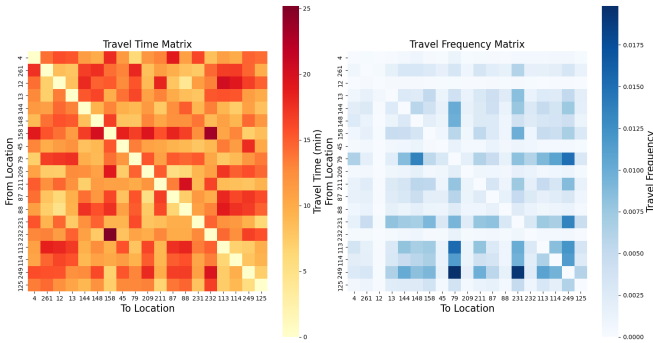
我们使用 2024 年 1 月至 6 月的纽约出租车出行数据，可从 NYC 出租车和豪华轿车委员会公开获取 (<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>)，以及 2023 年 1 月至 6 月的芝加哥出租车出行数据，可从芝加哥数据门户公开获取 (https://data.cityofchicago.org/Transportation/Taxi-Trips-2013-2023-/wrvz-psew/about_data)，生成真实的模拟场景。对于曼哈顿，我们的分析重点是在市中心区域内始发和终止的行程。对于芝加哥，我们重点关注中心区，包括 Loop 社区 (32 区) 和 18 个邻近社区。我们不是重播历史轨迹，而是从数据集中提取关键统计属性并使用它们来生成随机场景。

图 13 a 显示了曼哈顿市中心地区六个月内出租车出行的平均空间分布。根据这个分布，我们将高峰时段定义为从 12:00 a.m. 开始的时间间隔。至 10:00 p.m.。我们进一步计算每个 OD 对的经验频率，如图 13 b 所示。对于每个合成场景，旅行任务根据归一化的 OD 频率分布进行采样，从而保留在现实世界数据中观察到的真实空间和时间模式。

在图 14 a 中，我们显示了芝加哥中心区的类似统计数据。这些区域的高峰时间定义为 8:00 a.m. 起。至 7:00 p.m.。另请注意，出租车出行高度集中在出发地 - 目的地对 (32, 8) 和 (32, 28)，这是因为卢普区 (32 区) 是芝加哥的中央商务区，而近北区 (8 区) 和近西区 (28 区) 是毗邻卢普区的两个人口较多的区域。



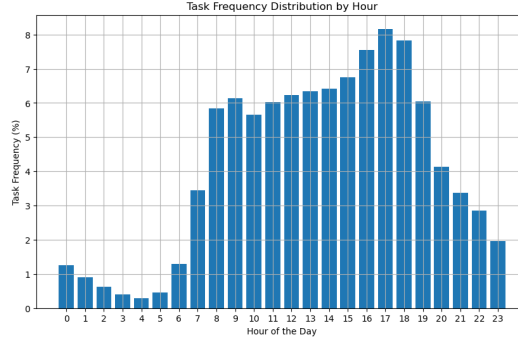
(a) 曼哈顿市中心平均任务分布



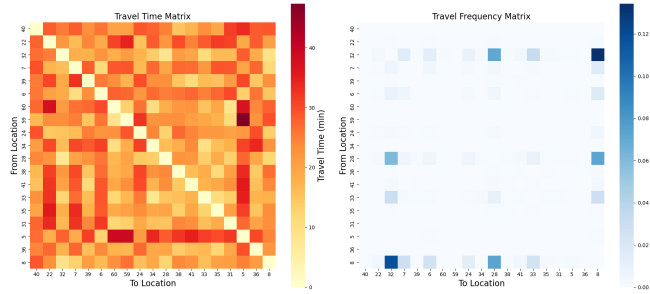
(b) 曼哈顿市中心出行时间和频率

图 13：曼哈顿市中心平均任务（乘客请求）分布及出行时间和频率统计

我们构建了两组，每组九个综合模拟场景，分别根据纽约 TLC 出行数据和芝加哥出租车出行数据的统计分布得出。每组都包含在乘客需求、出租车车队规模和请求方面共享相同配置的场景



(a) 芝加哥中心区平均任务分布



(b) 芝加哥市中心区出行时间及班次

图 1 4：芝加哥扩展中心区平均任务（乘客请求）分布及出行时间和频率统计

时间窗口，但每个城市的时空分布特征有所不同。每个场景由以下变量参数化：（1）乘客量

（P）：范围从 35 到 200 次请求，反映不同的需求强度。

（2）出租车车队规模（C）：600 至 100 辆，测试资源限制下的可扩展性。

（3）请求时间窗口（T）：600 s (10 min)、900 s (15 min) 或 1200 s (20 min)，是分配乘客请求的时间窗口。表 2、表 3 中场景 1 - 9 对应 P 35_C 60_T 600、P 65_C 80_T 600、P 100_C 100_T 600、P 50_C 60_T 900、P 100_C 80_T 900、P 150_C 100_T 900、P 70_C 60_T 1200、P 130_C 80_T 1200、P 200_C 100_T 1200。例如，P 200_C 100_T 1200

表示一个模拟场景，20 分钟时间窗口内发生 200 个乘客请求，系统中有 100 辆可用出租车，并且基于出发地 - 目的地分布随机生成请求。这些场景沿着这些维度对系统进行增量压力测试（表 2 和表 3），避免历史重播，同时保持对现实世界旅行动态的保真度。

A. 4 算法完整流程

完整的算法流程如算法 2 所示。

A. 5 动态系统的搜索空间

实时决策中的动态系统（例如，网约车调度）的特点是随时间变化的状态和顺序相互依赖的决策。与静态 OR

问题（例如，旅行商问题或装箱问题）不同，动态系统需要不断适应不断变化的环境，从而导致搜索空间呈指数级增长。 t 时刻的问题可以表示为： $S_t = \{V_t, P_t, C_t, D_t\}$ ，其中 S_t 是 t 时刻的系统状态，由车辆集 V_t （具有位置和状态的出租车）、乘客集 P_t （时间窗口内的上/下车位置的待处理乘客请求）、约束集 C_t （环境约束）和 t - D_t 之前的历史决策轨迹组成。

在每个决策步骤 t ，调度员在 εA_t 处选择一个动作，其中 A_t 表示动作空间（例如，为请求分配出租车以及每个出租车的访问顺序）。系统转换到新状态

算法 2: LLM - 优化器交互协议

要求: 群体规模 N 、控制间隔 Δt 、场景信息 U

确保: 不断发展的提示词群体和优化的调度策略

```
1: 用  $N$  个个体初始化种群  $P$  2: for Generation = 1 到  
max_Generations do 3: for round  $i$  = 1 to  
Popsize  $N$  do ▷可以同时触发 4: 父响应, 进化方式列表 =  
GETNEWINDIVIDUAL( $P$ ) ▷当前种群 5: 用环境数据初始化模拟器  $E0$   
6: for step  $t$  = 1 to  $T$  do ▷内部模拟循环 7:  $E_t$   
←GETSIMULATORSTATE() ▷当前车辆、请求等 8: if way[ $t$ ] =  
"way1" then ▷随机生成 9: 提示词  
←GENERATERANDOMPROMPT() 10: else if way[ $t$ ] =  
"way2" then ▷内存考虑 11:  
提示词←GENERATEREVISITEDPROMPT(父响应) 12: else  
▷("way3") 13: 提示词  
←GENERATEINNOVATEDPROMPT(父响应) 14: 结束 if 15:  
提示词 ←prompt $t$  + INJECTCONTEXT( $U$ ,  $E_t$ )  
▷添加场景和环境数据 16: 响应 ←QUERYLLM(prompt $t$ ) ▷Call  
API 17: 目标 ←PARSERESPONSE(response $t$ ) ▷提取 MILP  
目标 18: 解决一级分配 MILP: 19: 最小目标 20:  
 $y^*$ ←GUROBISOLVE(assign_model) 21: 解决二级排序 MILP:  
▷可以触发不同出租车同时行驶 22: 分钟  $P$ 
```

```
                
$$p = \max(0, \tau \cdot \text{拾取} - \tau \cdot \text{要求} \cdot p)$$
  
23:  $x^*, \tau^*$ ←GUROBISOLVE(sequence_model) 24:  
 $E_{t+1}$  ←UPDATESIMULATOR( $x^*, \tau^*, \Delta t$ ) ▷进化  $\Delta t$  秒  
25: 26 结束: fitness $i$  ←COMPUTEFITNESS( $E1, \dots, ET$ ) ▷系统总成本 27:  $P_{\text{new}} \leftarrow P_{\text{new}} + P_i$  ▷ $P_i = (E_t, \text{Objective}_t, \text{fitness})$  i for  $t \in T$  28: 结束 29:  $P$   
←SORTPOPULATION( $P, P_{\text{new}}$ ) ▷ 对组合人口进行排序并保留前  $N$  30:  
结束
```

S_{t+1} 基于 a_t 的作用, 由以下公式控制: $S_{t+1} = f(S_t, a_t)$ 。动态搜索空间 S 系统是有限范围 T 上所有可能的状态 - 动作序列的并集: $S = \bigcup_{t=0}^T \{(S_t, a_t)\}$ 。

每一步的动作空间 A_t 与车辆数量 $|V_t|$ 组合增长以及乘客请求的数量 $|P_t|$ 。

- 对于每一步乘客和出租车之间的分配, 其空间可以是 $O(|V| |P|)$ 。
- 假设分配给每辆出租车的 K 个请求, 则有效序列的数量为 $K!$, 其中 $|V|$ 出租车, 我们可以拥有 $O((K!) |V|)$ 。
- 给定 T 个步骤, 总搜索空间将增长为 $O((|V| |P| K! |V|)^T)$ 。

与传统的一次性 OR 问题相比, 这会导致极大的搜索空间, 使得 LLM 很难直接解决。虽然 LLM 不断发展, 并且当前的限制可能不会在未来的迭代中持续存在, 但搜索空间的复杂性需要一种混合方法, 将传统优化方法与 LLM 集成在分层框架中。因此, 我们将问题分为两个层次:

高层分配模型中的 LLM 驱动的目标设计 LLM 充当元优化器, 动态提出自适应目标来指导决策。生成的自适应目标函数 Φ_{h_t} 编码高层目标, 以当前状态 S_t 和历史信息 H_{t-1} 作为输入:

$$\begin{aligned} \text{最小 } \Phi_{H(t-1)}^h(t) &= \text{LLM}(S(t), \\ \text{s.t. } \sum_v y^{pv} &= 1, \quad y^{pv} \in \{0, 1\} \end{aligned}$$

一方面，分配层的变量 y^{pv} 缺乏对测序结果的直接可见性（例如，等待时间，路线效率），另一方面，手工制定的目标（例如，“最小化分配距离”）是短视的，因为它忽略了下游测序的影响。为了解决这一限制，LLM 动态生成自适应目标 $\Phi_{LLM}^l(t)$ ，隐式将分配引导至有利于低层测序的配置。

底层排序模型给定固定的分配计划，每辆出租车要解决一个路由问题来逐一拜访分配的乘客，目标是最小化乘客的总等待时间：

$$\begin{aligned} \text{最小 } \Phi_l^l(t) &= \tau(t) \\ \text{s.t. } g(S(t), \tau(t), \text{alt}) &\leq 0 \end{aligned}$$

其中 $\tau(t)$ 表示时间变量（例如，出租车到达时间）， alt 是低层动作，即路由变量。

A.6 直接建模方法

完整的出租车决策问题被表述为一个混合逻辑问题，其目标是最大限度地减少乘客等待时间，并结合初始汽车和乘客位置 / 时间分配、任务连接约束、车辆行驶动态和乘客登机动态等约束。我们考虑的网络被定义为有向图 $G = \{S, L\}$ ，其中 S 是网络中上车 / 下车站点的集合，其中任何站点 $i \in S$ 。 L 是任意两个站点之间的路线路径。多名乘客（出租车）可能共享相同的出发地和 / 或目的地。我们利用服务点集 $P(\{(p, O_p)\}, \{(v, S_v)\})$ ，对于出发地）和 $D(\{(p, D_p)\}, \{(v, E_v)\})$ ，对于目的地）来区分各个任务，同时保留乘客地理信息。

初始和最终车辆位置分配每次发生新的规划时，调度程序都需要知道出租车的初始位置，无论此时出租车处于空闲状态还是在道路上行驶，我们假设应提前知道该出租车到达下一个直接停靠点的剩余时间，分别如约束条件（3 a）和（3 b）所示。此外，出租车在服务完所有指定乘客后最终应返回接收点位置，如约束条件（3 c）所示。 X

$$i \in P_{xv} \quad (v, S_v), i = 1 \quad (3a)$$

$$AR_v(v, S_v) = \tau_v \quad (3b) \quad X$$

$$i \in P_{xv} \quad i, (v, E_v) = 1 \quad (3c)$$

其中， $x_{v, i, j}$ 表示是否为出租车 v 选择上下车对 i, j 。 $AR_v(i)$ 为接送服务 i 的车辆 v 的到达时间，其中 $i \in P$ 。

乘客分配限制 每个乘客最多只能搭乘一辆车，如（4）所示。 X

$$\sum_v y^{pv} = 1 \quad (4)$$

另外，每当出租车 v 被分配服务乘客 p 时，它会首先到达车站 O_p 接载乘客，然后前往车站 D_p 将乘客送至目的地。因此，运营出租车 v 应确认所选乘客 p 的边缘 OD 。

$$x_{v, (p, O_p), (p, D_p)} \geq y^{pv} \quad (5)$$

服务连接限制每当将一组服务（每个服务包含一个上车点和一个下车点）分配给出租车时，对于任何两项服务，出租车必须在开始下一项服务之前完成一项服务。换句话说，如果乘客 p 和 p' 都被分配到同一出租车 v ，则首先服务乘客 p 或首先服务乘客 p' 。该顺序反映为当前问题表述中典型车辆路径问题的守恒定律，如约束（6）中所捕获。

$$\begin{aligned} \forall v \in V, \forall i \in \mathcal{P}, \forall j \in \mathcal{D} \\ \sum_{i \in \mathcal{P}} x_{ij}^v &= q \in \mathcal{P} \quad x_{v \ j \ q} \quad (6a) \\ \sum_{j \in \mathcal{D}} x_{ji}^v &= k \in \mathcal{D} \quad x_{v \ i \ k} \quad (6b) \end{aligned}$$

另外，每位乘客必须由一辆出租车服务，如（4）所示，因此，任何一辆出租车只能前往每个上落客服务点一次，如下所述：

$$\begin{aligned} \forall v \in V, \forall i \in \mathcal{P}, \forall j \in \mathcal{D} \\ \sum_v i \in \mathcal{P} \quad x_{v \ i \ j} &= 1 \quad (7a) \\ \sum_v j \in \mathcal{D} \quad x_{v \ i \ j} &= 1 \quad (7b) \end{aligned}$$

到达和出发时间约束 如果为出租车 v 选择乘客 p 的路线边缘 OD ，则到达下车点的时间等于在起点上车的时间与 OD 行驶时间之和。此外，下车点的出发时间大于其到达时间。

$$\begin{aligned} x_{(p,Op),(p,Dp)}^v &= 1 \rightarrow \\ AR_v(p, Dp) &= DP_v(p, Op) + TROpDp \quad (8a) \\ DP_v(p, Dp) &\geq AR_v(p, Dp) \quad (8b) \end{aligned}$$

其中 $DP_v j$ 是车辆 v 前往下车服务 j 的出发时间，其中 $j \in \mathcal{D}$ 。 $A \rightarrow B$ 是一个逻辑约束，表示如果 A 为真，则意味着 B 。

另外，如果出租车 v 在乘客 p 之后立即为乘客 p' 提供服务，则乘客 p' 到达上车点的时间等于乘客 p 在下车点的出发时间与边缘行程时间之和。此外，上车点的出发时间应等于出租车到达时间和乘客到达 / 请求时间之间的最大值。

$$\begin{aligned} x_{(p,Ep),(p',Op')}^v &= 1 \rightarrow \\ AR_v(p', Op') &= DP_v(p, Dp) + TRDpOp' \quad (9a) \\ DP_v(p', Op') &= \max(AR_v(p', Op'), T_{p'}) \quad (9b) \end{aligned}$$

目标函数目标是最大限度地减少乘客的总等待时间，如下所示：

$$J = \sum_p \lambda_v WT_{pv} \quad (10)$$

其中等待时间 WT_{pv} 由以下约束得到，只有当乘客 p 被分配到出租车 v 时，相应的等待时间 WT_{pv} 才存在，否则应为 0：

$$y^{pv} = 1 \rightarrow WT^{pv} = DP_{(p,Op)}^v - T^p \quad (11a)$$

$$y^{pv} = 0 \rightarrow WT^{pv} = 0 \quad (11b)$$

虽然可以纳入平台利润或司机收入等替代目标来考虑更广泛的利益相关者影响，但这项工作的目的并不是设计一个全面或多方面的目标。相反，它重点关注 LLM 和优化器之间的交互，展示通过自然语言指定的高层目标如何演变为指导决策的低层目标。其他目标的扩展很简单，留待未来的工作。

上述提出的模型被表述为混合逻辑模型，结合了线性约束和逻辑约束（8）（9）和（11）。这些逻辑约束可以使用 big-M 方法 [45] 等价地转换为线性约束，详细公式在附录 A.7 中提供。尽管可以将该问题重新表述为 MILP 模型并直接使用 Gurobi 等商业求解器进行求解以获得最佳结果，但其大规模带来了巨大的计算挑战。例如，在出租车系统中，每次都会触发 80 辆出租车和 50 个乘客请求的场景，仅决策变量 $x_{v,i,j}$ 就导致 MILP 中有 200,000 (80 * 50 * 50) 个变量。考虑到实时操作所需的决策间隔很短，在可用时间范围内最优地求解如此大规模的 MILP 是不切实际的。因此，该问题通常被表述为顺序方法：首先，解决分配问题，将一部分乘客分配给一组出租车司机，然后并行解决旅行商问题，并为每辆出租车提供额外的时间限制。

A.7 逻辑约束转换

在 A.6 节中，问题被表述为混合逻辑动态模型，其中涉及逻辑约束例如、到达和出发时间约束（8）、（9）和等待时间约束（11）。所有这些逻辑约束都可以通过 big-M 方法转换为线性约束，如下所示。

命题 1 在模型中用不等式（12）替换逻辑约束（8）会得到相同的解。

证明：令 $M1$ 足够大，且满足 $M1 \geq \max\{\pm(AR_v(p, Dp) - DP_v(p, Op) - TR(Op, Dp)), \pm(AR_v(p, Dp) - DP_v(p, Dp))\}$ 对于 $v \in V, p \in P$ ，则约束（8）可重写为：

$$\forall v \in V, p \in P \quad AR_v(p, Dp) - DP_v(p, Op) - TR(Op, Dp) \leq M1(1 - x_{v,p}(p, Op)(p, Dp)) \quad (12a)$$

$$-AR_v(p, Dp) + DP_v(p, Op) + TR(Op, Dp) \leq M1(1 - x_{v,p}(p, Dp)(p, Op)) \quad (12b)$$

$$AR_v(p, Dp) - DP_v(p, Dp) \leq M1(1 - x_{v,p}(p, Op)(p, Dp)) \quad (12c)$$

$$-AR_v(p, Dp) + DP_v(p, Dp) \leq M1(1 - x_{v,p}(p, Op)(p, Dp)) \quad (12d)$$

命题 2 在模型中用不等式（13）替换逻辑约束（9）会得到相同的解。

证明：令 $M2$ 足够大，且满足 $M2 \geq \max\{\pm(AR_v(p', Op') - DP_v(p, Dp) - TR(Dp, Op')), \pm(AR_v(p', Op') - DP_v(p', Dp')), \pm(AR_v(p', Op') - DP_v(p, Dp) - TR(Dp, Op')), \pm(AR_v(p', Op') - DP_v(p', Dp'))\}$ 对于 $v \in V, p \in P$ ，则约束（9）可重写为：

$$-AR_v(p', Op') + DP_v(p, Dp) + TR(Dp, Op') \leq M2(1 - x_{v,p}(p, Ep)(p', Dp')) \quad (13b)$$

$$DP_v(p', Dp') - AR_v(p', Op') \geq -M2(1 - x_{v,p}(p, Ep)(p', Dp')) \quad (13c)$$

$$DP_v(p', Dp') - TR(p', Dp') \geq -M2(1 - x_{v,p}(p, Ep)(p', Dp')) \quad (13d)$$

命题 3 在模型中用不等式（14）替换逻辑约束（11）会得到相同的解。

证明：令 $M3$ 足够大，且满足 $M3 \geq \max\{\pm(WT_{pv} - DP_v(p, Op) - T_p), \pm(WT_{pv})\}$ 对于 $v \in V, p \in P$ ，则约束（11）可重写为：

$$WT_{pv} - DP_v(p, Op) - T_p \leq M3(1 - y_{pv}) \quad (14a)$$

$$-WT_{pv} + DP_v(p, Op) + T_p \leq M3(1 - y_{pv}) \quad (14b)$$

$$WT_{pv} \leq M3y_{pv} \quad (14c) \quad WT_{pv} \geq -M3y_{pv} \quad (14d)$$

A . 8 动态模拟器环境设置

在模拟器中, k 时刻交通网络的状态被简化为 $y(k) = \{d_i(k), tarr_i(k), i \in I\}$, 其中 i 是出租车的索引, I 是所有出租车的集合, $d_i(k)$ 是第 i 辆出租车在 k 时刻行驶到的位置, $tarr_i(k)$ 是第 i 辆出租车到达该位置的时间。模拟中的状态和任务更新遵循以下约定:

1): 正在执行的任务不会被重新分配。

2): 如果出租车没有分配任务, 它将停留在最后一个任务的目的地。

优化器发出的命令是交通系统中每辆出租车的一组任务序列。每个任务有四个参数: 出发地 o 、目的地 d 、乘客到达时间 $tstart$ 、出租车出发时间 $tdep$ 。我们将优化器在 k 时刻发出的第 i 辆出租车的命令表示为 $uopt_i(k)$, 并将正在执行的命令表示为 $ui(k)$ 。当优化器向出租车发出命令时, 命令 $ui(k)$ 将被设置为 $uopt_i(k)$ 。命令 $ui(k)$ 定义为 $ui(k) = \{task_i, 0(k), \dots, task_i, Ni(k)\}$, 其中 $task_i, j(k) = (oi, j(k), di, j(k), tarr_i, j(k), tstart_i, j(k), tdep_i, j(k))$ 。分别收集出发地、目的地、出租车到达时间、乘客到达时间和出租车从任务出发地出发的时间。对于系统状态 $y(k)$ 和命令 $u(k) = \{ui(k), i \in I\}$, 状态更新为 $y(k+1) = f(y(k), u(k), k)$, 其中对于每个 k , $f(\cdot, \cdot, \cdot)$ 在算法 3 中给出。当接收到新计算的 $uopt(k) = \{uopt_i(k), i \in I\}$ 时, 状态和命令应按 4 中的方式更新。

Algorithm 3 State Transition Model

```

1: 输入: 当前状态  $y(k)$ , 命令  $u(k)$  2: 输出: 下一状态  $y(k+1)$ 
3:  $k \leftarrow k + 1$  4: 对于每个出租车  $i \in I$  执行 5:
if  $ui(k) \neq \emptyset$  then 6:  $(oc, dc, tarr_c, tstart_c, tdep_c) \leftarrow head(ui(k))$  7: if
 $k = tarr_i$  then 8: if  $di = oc$  then 9:
更新目的地:  $di \leftarrow dc$  10: 更新到达时间:  $tarr_i \leftarrow tdep_c + TRoc, dc$  11: else 12:  $pop(ui(k))$  13:
if  $ui(k) \neq \emptyset$  then 14:  $(oc, dc, tarr_c, tstart_c, tdep_c) \leftarrow head(ui(k))$  15:
转到下一个任务:  $tarr_i \leftarrow \max\{k, tstart_c\} + TRdi, oc, di \leftarrow oc$  16: end if 17: end if
18: end if 19: else 20: 将出租车  $i$  标记为空闲
21: end if 22: end for

```

Algorithm 4 Command Update Protocol

```

1: 输入: 优化命令  $uopt(k)$ , 当前状态  $y(k)$ 
2: 输出: 更新命令  $u(k+1)$  3: 对于每个出租车
 $i \in I$  执行 4: if  $ui(k) = \emptyset$  then 5:
 $ui(k+1) \leftarrow uopt_i(k)$  6: else
7: 合并命令:  $ui(k+1) \leftarrow append(ui(k), uopt_i(k))$ 
8: 结束如果 9: 结束

```

A . 9 提示词工程

在本节中，我们提供了我们的方法中使用的提示词策略的详细信息。

A . 9 . 1 场景提示词组成

图 1 5 说明了 3 . 2 . 1 节中介绍的场景提示词结构，其中场景提示词分别由 P s y s 、 P g e o 和 P m o d e l 组成。

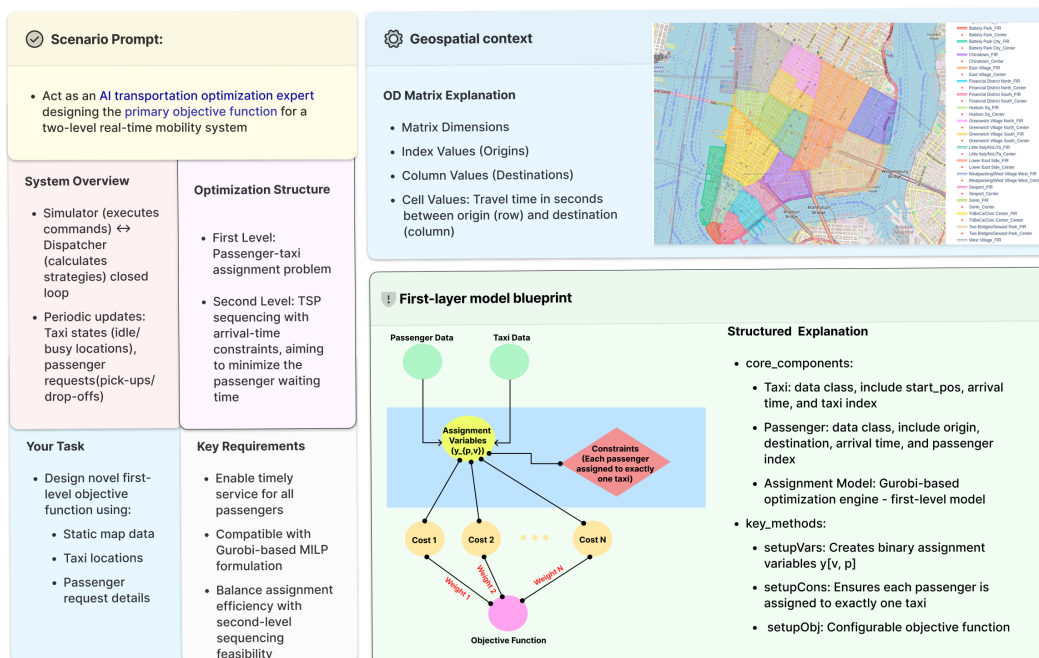


图 1 5：向 L L M 提供的场景提示词结构，包括系统架构和角色定义（左图）、地理空间内容（右上图）、第一层模型蓝图（右下图）

系统架构和角色定义（P s y s）系统架构和角色定义将 L L M 确立为自适应目标设计器，而不是端到端求解器。

系统架构：我们简要概述了系统架构，包括模拟器和调度程序之间的通信，调度问题的两级分解，即 $M a s s i g n : \min_y L a s s i g n(y | \theta_{LLM})$ 和 $M r o u t e : \min_x L r o u t e(x | y^*)$ 。

角色规范：定义 L L M 的角色，以使用基于模板的生成将其输出限制为与 G u r o b i [4 2] 求解器 A P I 兼容的目标函数组件。

地理空间背景 P g e o

由于问题与网约车服务有关，因此纳入地理空间背景至关重要。因此，我们向 L L M 提供测试环境详细信息。利用公开的纽约或芝加哥出租车数据集，我们的分析重点关注曼哈顿市中心或芝加哥市中心扩展区的 1 9 个区域。静态地图数据表示为 O D 矩阵 W O D，通过曼哈顿区域图 $G = (V, E, W O D)$ 编码空间语义，其中 $W O D[i, j] = t_{ij} \in R^+, \forall (v_i, v_j) \in E$ 。

第一层模型蓝图 P m o d e l 暴露第一层分配模型类结构以指导 L L M 兼容的目标设计。该类结构为 L L M 提供了一个工作框架，确保其输出与整体模型设计一致。通过访问类结构，L L M 可以了解不同变量之间的关系，并设计考虑这些关系的目标函数。

A.9.2 和声搜索提示词指令

正如第 3.2.3 节中所讨论的，我们的进化算子采用结构化指令块来指导基于 LLM 的提示词优化。具体说明如下：

启发式改进 (W2) 指令块：专注于增量目标细化：

- (a) 时间协调：纳入出租车 - 乘客到达时间协调。
- (b) 资源权重：自适应出租车利用系数
- (c) 结构保留：保留 50% 的遗留目标组成部分。

创新生成 (W3) 指令块：促进范式级创新：

- (a) 强调目标：设计第一层决策 $y_{v,p}$ ， p 最大限度地减少预期的第二层等待：

$$\min_{p \in P} \max_{p \in P} \text{DP}(2nd)_{p - \text{tarr}, 0, y_{v,p}, i}$$

- (b) 多时域优化：当前 / 未来状态联合考虑
- (c) 动态权重自适应：随时间变化的优先级系数

A.9.3 Gurobi 格式限制

为了确保目标函数与 Gurobi 兼容，引入了额外的约束以增强其可行性。施加了两种类型的限制：1. 可接受的术语：这些约束定义了目标函数的完整结构，确保函数中的所有组件都遵守类要求。2. 表达式构造规则：指定 Gurobi 表达式框架中允许的有效运算和转换的指南，以保持求解器兼容性。

允许的组件 生成的目标函数必须仅使用以下类元素，表 7 中也总结了这些元素：

- **Decision variables:** Binary assignments: $\text{self.y}[v,p] \in \{0,1\}$
 - 静态参数：
 - 距离矩阵：`self.distMatrix[o][d]` Big-M 常数：`self.M`
 - 出租车状态 (只读)：
 - 当前位置：`self.taxi[v].start_pos`
 - 可用时间：`self.taxi[v].arrival_time`
 - 乘客状态 (只读)：
 - 出发地 / 目的地：`self.passenger[p].origin`、`self.passenger[p].destination`
 - 请求时间：`self.passenger[p].arrTime`

表 7：模型规范映射

概念	数学形式	代码实现
赋值变量	$y_{v,p}$	<code>self.y[v,p]</code>
出租车可用性	τ	<code>self.taxi[v].arrival_time</code>
乘客请求时间	τ_{req}	<code>self.passenger[p].arrTime</code>
行程时间矩阵	$T_{o,d}$	<code>self.distMatrix[o][d]</code>
加权总和	$\sum(w * c)$	<code>sum(w * c for w, c in zip(weights, costs))</code>

表达式构建规则 所有目标必须遵守：

1. 结构要求：成本存储在列表 `costs` 中（1 - 5 个元素）列表 `weights` 中的权重（长度匹配 `costs`）最终目标：`sum(w*c for w, c in zip(weights, costs))`
Gurobi 表达式规则：

- Use `**gb.quicksum()/gb.max()/gb.abs()` only when containing variables
- Use `**sum()/max()/abs()` when working with parameters
- Quadratic terms via variable multiplication (`y[v,p]*y[v,q]`)
- Never multiply Gurobi variables with Gurobi expressions
 - Invalid: `y[v,p] * gb.max_(expression_with_vars, 0)`
- Never use Python if/else with Gurobi variables/expressions

生成协议向 LLM 提供以下模板以进行有效的目标构建：

```
def dynamic_obj_func(self):
    cost1 = [Proper Gurobi expression]
    cost2 = [Proper Gurobi expression]
    # Add more components as needed

    # Custom weights (match costs length)
    weights = [
        [Your custom weight 1],
        [Your custom weight 2],
        # Add matching weights
    ]
    objective = sum(w*c for w,c in zip(weights, costs))
    self.model.setObjective(objective, gb.GRB.MINIMIZE)
```

A.9.4 LLM 响应示例

图 17 和图 18 显示了由所提出的方法生成的四个 LLM 响应示例。第一层模型蓝图定义了默认目标函数，作为 LLM 的基础启发式方法，如图 16 所示。列出的四个目标函数变体捕获了不同的优化策略：

```
Default Objective Code
def default_dynamic_obj_func(self):
    cost1 = gb.quicksum(gb.quicksum((self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin]*self.y[v,p]) for p in self.passenger.keys() for v in self.taxi.keys()
    cost2 = gb.quicksum(gb.quicksum((self.distMatrix[self.taxi[v].start_pos][self.passenger[p].destination]*self.y[v,p]) for p in self.passenger.keys() for v in self.taxi.keys()
    cost3 = gb.quicksum(gb.quicksum((max((self.taxi[v].arrival_time - self.passenger[p].arr_time, 0) * self.y[v,p]) for p in self.passenger.keys() for v in self.taxi.keys()
    cost4 = gb.quicksum(gb.quicksum((self.y[v,p]) for p in self.passenger.keys() * gb.quicksum((self.y[v,p]) for p in self.passenger.keys() for v in self.taxi.keys()
    costs = [cost1, cost2, cost3, cost4]
    weights = [1, 1, 1, 100]
    self.model.setObjective(gb.quicksum(w*c for w,c in zip(costs, weights)), gb.GRB.MINIMIZE)
```

图 16：第一层模型蓝图中使用的默认目标代码

具有动态惩罚的加权多目标 如图 17 左图所示，与默认情况下的静态 `weights` 不同，此变体引入了动态惩罚（例如，`cost2` 中的时间尺度空闲出租车优先级、`cost5` 中的未来可用性调整）并放大了优先级 `weights`（例如，等待时间 80，负载不平衡 1000）。通过根据实时条件（例如，出租车到达与乘客请求时间）动态调整处罚，它可以自适应地平衡短期效率和长期车队可用性，解决默认情况下处理依赖时间的场景的刚性问题。

具有重新分配惩罚的二次负载平衡虽然默认通过 `cost4` 惩罚负载不平衡，但此变体（如右图图 17 所示）通过与平均乘客 / 出租车比率（`load_balance`）的平方偏差明确强制执行公平性。此外，该变体还包含二次重新分配惩罚（`cost3`），以最大限度地减少频繁的出租车重新分配。这种双重关注有效地减少了重新分配流失并增强了公平性，解决了默认模型在执行平衡分配方面的局限性。

通过负载平衡进行繁忙出租车惩罚 该变体（如图 18 的左图所示）取代了默认的简单等待时间惩罚，使用 $>$ 运算符（`cost4`）来检查出租车的可用性是否先于乘客的请求，直接对繁忙出租车的分配进行惩罚。该变体将完整行程持续时间纳入 `cost2`，超出了默认模型仅关注接载时间的范围。此外，它还保持二次负载平衡，以更有效地分配需求。通过考虑实时可用性和总行程工作量，这种方法可以防止对已占用的出租车进行过多的分配，解决了默认模型的一个关键限制。

下车 - 上车链接的顺序分配与默认的孤立行程模型不同，此变体（如图 18 右图所示）在 `cost3` 中引入了顺序链接机制，惩罚了乘客之间的下车到上车的行程时间（ $y[v, p] * y[v, p-1]$ ）。它还增加了双向等待时间惩罚（`cost1` 表示出租车延误，`cost5` 表示乘客提前），而默认值仅惩罚出租车延误。通过优化行程顺序并考虑等待约束，它提高了车队利用效率，这是对默认单行程优化方法的改进。

Objective Description Objective function prioritizes minimizing passenger waiting time by dynamically adjusting weights, encouraging the use of idle taxis, and considering future locations and arrival times to ensure efficient taxi assignments.	Objective Description This objective function prioritizes passenger waiting time and taxi idle travel distance, while ensuring load balancing across taxis to prevent over-assignment and underutilization. It introduces a quadratic penalty to discourage reassignment of taxis, encouraging full utilization of available taxis. The weights are adjusted to balance the importance of each component effectively.
Objective Code <pre>def default_dynamic_obj_func(self): # Cost 1: Passenger waiting time with dynamic penalty based on arrival time difference cost1 = gb.quicksum(max(self.taxi[v].arrival_time - self.passenger[p].arrTime, 0) * 80 * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) # Cost 2: Taxi movement time from current location to passenger origin with idle taxis priority cost2 = gb.quicksum(self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin] * (10 + (self.taxi[v].arrival_time - self.passenger[p].arrTime) / 3600) * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) # Cost 3: Passenger ride time scaled by future availability of Taxi cost3 = gb.quicksum(self.distMatrix[self.passenger[p].origin][self.passenger[p].destination] * (1 + (self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin] / 1000)) * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) # Cost 4: Load balancing penalty to prevent overloading, encouraging even distribution cost4 = gb.quicksum((gb.quicksum(self.y[v, p] for p in self.passenger.keys()) ** 2) * 1000 for v in self.taxi.keys()) # Cost 5: Future availability penalty adjusted for expected future tasks cost5 = gb.quicksum((self.taxi[v].arrival_time + self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin]) * (1 - (self.taxi[v].arrival_time - self.passenger[p].arrTime) / 3600) * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) costs = [cost1, cost2, cost3, cost4, cost5] weights = [80, 10, 1, 1000, 30] # Set the objective function with adjusted weights objective = sum(w * c for w, c in zip(weights, costs)) self.model.setObjective(objective, gb.GRB.MINIMIZE)</pre> Weighted Multi-Objective with Dynamic Penalties	Objective Code <pre>def default_dynamic_obj_func(self): # Cost component 1: Passenger waiting time cost1 = gb.quicksum(max(self.taxi[v].arrival_time - self.passenger[p].arrTime, 0) * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) # Cost component 2: Taxi idle travel distance from current position to passenger origin cost2 = gb.quicksum(self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin] * self.y[v, p] for v in self.taxi.keys() for p in self.passenger.keys()) # Cost component 3: Quadratic penalty for reassigning taxis (discourage) cost3 = gb.quicksum((gb.quicksum(self.y[v, p] for p in self.passenger.keys())) * (gb.quicksum(self.y[v, p] for p in self.passenger.keys())) for v in self.taxi.keys()) # Cost component 4: Load balancing to ensure even distribution across taxis load_balance = sum((sum(self.y[v, p] for p in self.passenger.keys()) - (len(self.passenger) / len(self.taxi)))**2 for v in self.taxi.keys()) costs = [cost1, cost2, cost3, load_balance] weights = [3, 1, 150, 20] # Build and set the objective function objective = sum(w * c for w, c in zip(weights, costs)) self.model.setObjective(objective, gb.GRB.MINIMIZE)</pre> Quadratic Load Balancing with Reassignment Penalty

图 17：LLM 响应示例，包括动态惩罚变体（左图）和二次负载平衡变体（右图）

Objective Description Improved objective function considering arrival times for better taxi utilization and passenger waiting time balancing. Adjusted weights to prioritize waiting time and quadratic load balancing.	Objective Description An improved objective function that considers taxi arrival times, passenger request times, and optimizes for taxi efficiency while balancing assignment to reduce waiting times and travel costs.
Objective Code <pre>def default_dynamic_obj_func(self): # Cost1: Waiting Time Penalty for Passengers cost1 = gb.quicksum(gb.quicksum(max(self.taxi[v].arrival_time - self.passenger[p].arrTime, 0) * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) # Cost2: Taxi Travel Time from Current Position to Passenger Origin + Trip Duration cost2 = gb.quicksum(gb.quicksum((self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin] + self.distMatrix[self.passenger[p].origin][self.passenger[p].destination]) * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) # Cost3: Quadratic Term to Balance Load Across Taxis cost3 = gb.quicksum((gb.quicksum(self.y[v, p] for p in self.passenger.keys())) ** 2 for v in self.taxi.keys()) # Cost4: Penalty for Using Busy Taxis cost4 = gb.quicksum(gb.quicksum((self.taxi[v].arrival_time > self.passenger[p].arrTime) * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) # Weighting Coefficients weights = [200, 1, 800, 50] # Combine Costs total_cost = sum(w * c for w, c in zip(weights, [cost1, cost2, cost3, cost4])) self.model.setObjective(total_cost, gb.GRB.MINIMIZE)</pre>	Objective Code <pre>def default_dynamic_obj_func(self): # Cost1: Taxi waiting time (arrival before passenger request) cost1 = gb.quicksum(gb.quicksum(max(self.taxi[v].arrival_time - self.passenger[p].arrTime, 0) * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) # Cost2: Travel time from taxi start to pickup cost2 = gb.quicksum(gb.quicksum(self.distMatrix[self.taxi[v].start_pos][self.passenger[p].origin] * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) # Cost3: Dropoff-to-pickup travel time adjustment cost3 = gb.quicksum(gb.quicksum(self.distMatrix[self.passenger[p].destination][self.passenger[p_].origin] * self.y[v, p] * self.y[v, p_] for p_ in self.passenger.keys() if p != p_) for v in self.taxi.keys()) # Cost4: Passenger assignment imbalance penalty cost4 = gb.quicksum((gb.quicksum(self.y[v, p] for p in self.passenger.keys())) ** 2 for v in self.taxi.keys()) # Cost5: Passenger waiting time (request to pickup) cost5 = gb.quicksum(gb.quicksum(max(self.passenger[p].arrTime - self.taxi[v].arrival_time, 0) * self.y[v, p] for p in self.passenger.keys()) for v in self.taxi.keys()) costs = [cost1, cost2, cost3, cost4, cost5] weights = [1, 1, 1, 50, 2] objective = sum(w * c for w, c in zip(weights, costs)) self.model.setObjective(objective, gb.GRB.MINIMIZE)</pre>

图 18：LLM 响应示例，包括繁忙出租车变体（左图）和顺序分配变体（右图）